

DD: RLM-CFG-01 — HomePass Configuration

Developer implementation guide: DD_RLM-CFG-01-HomePass_Configuration-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_RLM-CFG-01-HomePass_Configuration-v1.0.md.

Verdict: ■ New (no Franchise / TechRegion / HomePass entities in TZ Sophix; KE Broadhub has them but they are not consolidated as a clean catalog with a status state machine, maker-checker, and event publishing) **Wave: 4 | Section A:** Configuration & Catalogs

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/09_catalogs/DD_RLM-CFG-01-HomePass_Configuration-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	homepass-status-transition
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/franchises/{id}
- DELETE /api/homepass-status-codes/{id}
- DELETE /api/homepasses/{id}
- DELETE /api/house-types/{id}
- DELETE /api/network-nodes/{id}
- DELETE /api/tech-contractor-skills/{id}
- DELETE /api/tech-contractors/{id}
- DELETE /api/tech-regions/{id}

- DELETE /api/tech-regions/{id}/contractors/{contractorId}
- GET /api/franchises
- GET /api/franchises/{id}
- GET /api/homepass-status-codes
- GET /api/homepass-status-codes/{id}
- GET /api/homepasses
- GET /api/homepasses/eligible
- GET /api/homepasses/eligible?franchiseRef=...&country=...®ionL1=...&roadName=...&buildingNumber=...&buildingName=...&apartmentNumber=...
- GET /api/homepasses/map?bbox=W,S,E,N&zoom=N&statusFilter=...&franchiseRef=...&accessTechnology=...
- GET /api/homepasses/map?bbox=west,south,east,north&zoom=N&statusFilter=...&franchiseRef=...
- GET /api/homepasses/transitions/pending
- GET /api/homepasses/{id}
- GET /api/homepasses/{id}/eligible-contractors?skill=...
- GET /api/homepasses/{id}/eligible-contractors?skill=INSTALLATION&status=ACTIVE
- GET /api/homepasses/{id}/eligible-contractors?skill=X
- GET /api/homepasses/{id}/transitions
- GET /api/house-types
- GET /api/house-types/{id}
- GET /api/network-nodes
- GET /api/network-nodes/{id}
- GET /api/tech-contractor-skills
- GET /api/tech-contractor-skills/{id}
- GET /api/tech-contractors
- GET /api/tech-contractors/{id}
- GET /api/tech-regions
- GET /api/tech-regions/{id}
- GET /api/tech-regions/{id}/contractors
- GET /map
- PATCH /api/franchises/{id}
- PATCH /api/homepass-status-codes/{id}
- PATCH /api/homepasses/{id}
- PATCH /api/homepasses/{id}/network-path/append
- PATCH /api/homepasses/{id}/network-path/ports
- PATCH /api/house-types/{id}
- PATCH /api/network-nodes/{id}
- PATCH /api/tech-contractor-skills/{id}
- PATCH /api/tech-contractors/{id}
- PATCH /api/tech-regions/{id}
- POST /api/franchises

- POST /api/franchises/{id}/activate
- POST /api/franchises/{id}/retire
- POST /api/homepass-status-codes
- POST /api/homepass-status-codes/{id}/activate
- POST /api/homepass-status-codes/{id}/retire
- POST /api/homepasses
- POST /api/homepasses/bulk-import
- POST /api/homepasses/hp_01HX/transitions
- POST /api/homepasses/hp_01HX/transitions/cam_proc_01HX/approve
- POST /api/homepasses/{id}/clone
- POST /api/homepasses/{id}/enrich-from-geo
- POST /api/homepasses/{id}/transitions
- POST /api/homepasses/{id}/transitions/{processInstanceId}/approve
- POST /api/homepasses/{id}/transitions/{processInstanceId}/reject
- POST /api/house-types
- POST /api/house-types/{id}/activate
- POST /api/house-types/{id}/retire
- POST /api/network-nodes
- POST /api/network-nodes/bulk-import
- POST /api/network-nodes/{id}/activate
- POST /api/network-nodes/{id}/retire
- POST /api/tech-contractor-skills
- POST /api/tech-contractors
- POST /api/tech-contractors/{id}/activate
- POST /api/tech-contractors/{id}/retire
- POST /api/tech-regions
- POST /api/tech-regions/{id}/activate
- POST /api/tech-regions/{id}/contractors
- POST /api/tech-regions/{id}/retire
- POST /enrich-from-geo
- POST /enrich-from-geo,
- POST /transitions

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- billing-rules
- end-approved
- end-approved-auto
- end-rejected
- enrich-from-geo
- field-ops
- geo-client
- geo-gw
- gw-needs-approval
- gw-outcome
- homepass-construction
- homepass-rules
- homepass-status-transition
- homepass-transition-pending
- iptv-gw
- notification-gw
- place-details
- provisioning-gw
- reverse-geocode
- start-transition
- task-approve-transition
- transition-approved
- transition-rejected
- voice-gw

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- HomePassAddressCorrected
- HomePassConflictException
- HomePassGeoEnriched
- HomePassNotFoundException
- HomePassReachedSellable
- HomePassService
- HomePassStatusChanged
- HomePassStatusCodeUpdated
- HomePassTransitionRejected
- HomePassUpdated

- HomePassValidationException

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

8. Readability Split For Implementation Teams

This DD is intentionally detailed because HomePass combines sales coverage, topology, contractor geography, status governance, GIS/reference data, and fulfillment-facing service endpoints. For implementation, do not assign the whole file to one developer as one work item. Split delivery into these workstreams while keeping this DD as the canonical source until the physical source split is produced:

Workstream	Sections to read	Implementation owner	Output
RLM-CFG-01A Core HomePass Catalog	Purpose, scope, homepass, homepass_status_code, house_type, status workflow, HomePass APIs.	HomePass backend dev.	Core catalog schema, CRUD/search APIs, status transition APIs.
RLM-CFG-01B Coverage And Topology Reference	network_node, network_path, service_management_endpoints, GIS integration, design map imports.	Network/GIS backend dev.	Network node hierarchy, service endpoint model, topology import/search APIs.

Workstream	Sections to read	Implementation owner	Output
RLM-CFG-01C Territory And Contractor Mapping	franchise, tech_region, tech_contractor_skill, tech_contractor, tech_region_contractor, HomePass franchise/tech-region links.	Field operations backend dev.	Territory assignment, contractor skill/region reference APIs.
RLM-CFG-01D Rules And Workflow	Drools rules, Camunda status transition, maker-checker, events.	Workflow/rules dev.	DRL packages, BPMN definitions, Kafka events, transition validation tests.

Recommended implementation order:

1. Implement Core HomePass Catalog tables and read APIs.
2. Add status code catalog and Camunda-backed status transition APIs.
3. Add network node and topology/path model.
4. Add territory and contractor-region reference data.
5. Add GIS/import tooling.
6. Add Drools validation packages and transition event publication.
7. Expose frontend/admin screens only after the relevant APIs are stable.

Performance rule: HomePass search endpoints must be indexed for operator, status, serviceability, geo/region, and HomePass reference. GIS/PostGIS is optional for MVP; indexed bounding-box or region-based lookup is acceptable for the first Kenya deployment if the future PostGIS boundary in this DD is preserved.

Verdict: ■ New (no Franchise / TechRegion / HomePass entities in TZ Sophix; KE Broadhub has them but they are not consolidated as a clean catalog with a status state machine, maker-checker, and event publishing)

Wave: 4 | **Section A:** Configuration & Catalogs

A — Target

1. What this process is about

This DD covers the three entities that describe **where Wananchi can sell and serve customers**. They are all owned by a new homepass module.

- A **Franchise** is a top-level commercial / operational region. Wananchi KE has roughly six franchises (e.g., Nairobi 002). The Franchise drives which sales team covers an address, which contractor pool installs, and which per-franchise price applies. A Franchise has a real geographic footprint (a polygon on the map), captured here as optional GeoJSON.
- A **TechContractorSkill** is a configurable skill type used to describe what kind of work a contractor is qualified to do: INSTALLATION, MAINTENANCE, SUPPORT, B2B, etc. Admins manage this as a catalog so new skills can be added without code change. Skills feed two downstream concerns: installation routing (FUL-03 picks a contractor with INSTALLATION skill) and ticketing (ASR-01 routes a ticket to a contractor with SUPPORT skill). Skills are properties of contractors, NOT of regions.
- A **TechContractor** is a contractor company or team that performs network or customer-facing work. It carries a non-empty `skills[]` array (e.g., [INSTALLATION, MAINTENANCE] for a fiber install crew; [SUPPORT] for a helpdesk team). A TechContractor is assigned to one or more TechRegions — the geography it serves.
- A **TechRegion** is a pure technical / operational geographic zone. Nairobi has roughly 8–9 TechRegions. **TechRegions are NOT structurally inside Franchises** — they are independent registries. A TechRegion is served by **one or more TechContractors** (via a join), and the `*skills` available in that TechRegion* are the union of the assigned contractors' skills. This was a deliberate v1 design choice: the commercial team manages Franchises, the technical team manages TechContractors and the regions they serve, and we don't want one to block the other. A TechRegion also has an optional GeoJSON footprint and an optional coverage polygon (the area within it that has GPON / HFC reach).
- A **HomePass** is a single addressable termination point — one apartment, one home, one shop. It is the addressable unit of coverage: a customer can only be onboarded at an address that exists as a HomePass with an eligible status. A HomePass is linked to **one or more Franchises** (a building in a commercial overlap zone, or B2B + B2C franchises both covering the address, may yield several refs) and **one or more TechRegions** (e.g., a residential TechRegion for the building's main service, plus a separate TechRegion if part of the building is in another administrative area). The HomePass exposes a primitive — "given a required skill, which contractors are eligible to work this address" — by walking HomePass → TechRegions → assigned Contractors filtered by skill; the actual WO routing logic that consumes this

primitive (slot capacity, scheduling, multi-hop role transfers) lives in the WO DD. The HomePass also carries (1) the **access technology** that physically reaches the address (GPON / HFC / COPPER / FTTH / WIRELESS — what cable is in the ground / on the pole), (2) the **services supported** derived from the network path (DATA, VOICE, IPTV_MULTICAST — what the HomePass can carry at its physical termination; cloud / OTT services consumed over the customer's DATA connection are NOT listed here — they're Service-level concerns on the PLM-CFG-01 / PLM-CFG-08 side. A phone-only POTS customer would have a HomePass with `accessTechnology = COPPER` and `servicesSupported = [VOICE]`), (3) the **network path** — the ordered chain of network nodes AND ports (OLT + port, splitter + port, FAT + port, ONT serial, ...) the address sits behind, each node a reference into the `network_node` catalog, and (4) its current status in the network construction lifecycle.

This DD covers the **configuration / catalog** dimension of these three entities: create, read, update, soft-delete, status transitions on HomePass, bulk import, plus audit. It also covers the maker-checker control on sensitive HomePass status transitions and the auto-notify-leads hook when a HomePass becomes RFS.

RLM-CFG-01 scope. This DD covers:

- Franchise catalog (CRUD + DRAFT/ACTIVE/RETIRED lifecycle, mirror of Service catalog pattern), with optional `boundaryGeoJson` polygon
- TechContractorSkill catalog (small CRUD admin: code + name + description; configurable list used by TechContractor and by downstream routing in FUL-03 and ASR-01)
- TechContractor catalog (CRUD + DRAFT/ACTIVE/RETIRED lifecycle, with non-empty array of skill refs); TechContractor ↔ TechRegion many-to-many join
- TechRegion catalog (CRUD + DRAFT/ACTIVE/RETIRED lifecycle, **independent of Franchise**, no skills column — skills come from per-assignment `tech_region_contractor.skills[]`), with optional `boundaryGeoJson` polygon
- HomePass entity: address + access technology + services supported + network path (with ports) + Franchise refs (many) + TechRegion refs (many) + status, plus CRUD + bulk import
- HomePass status code catalog (per-deployment configurable; default seed NPL / NSN / RFS / WAI / ACT / RETIRED with semantic flags — see Section 1 glossary). The **transitions** between codes are owned by a configurable Camunda workflow (see "Status workflow" section); the DD does NOT hardcode which transitions are valid
- The contract that any deployment's HomePass-status-transition workflow must honor (record proposals in the audit ledger, emit `HomePassStatusChanged` on every status flip, emit `HomePassReachedSellable` once per HomePass on its first transition into a sellable status)
- Two reference BPMN workflows shipped with the module: a **strict maker-checker** workflow (KE default — approval required for RFS+) and a **passthrough admin-driven** workflow (alternative — no approval, transitions apply immediately)
- Auto-notify hook on → RFS first transition (the event hook; the actual notification delivery is in `notification-gw`)
- The HomePass **network path model**: an ordered chain of network nodes WITH ports (OLT + port, splitter + port, FAT + port, ONT serial, HEADEND, distribution node, amplifier, line extender, voipswitch, ...) recording the technical route to the address. Supports both "fiber pre-deployed" (KE — full path with ports known at HomePass time) and "fiber on-customer-install" (e.g., SN — only upstream nodes known at HomePass time, ports captured by fulfillment during install) regions through a `captureMode` field.
- A pre-computed `serviceManagementEndpoints` lookup naming which node AND port the gateways must address per service family (data, voice, `iptv_multicast`, monitoring). Read by provisioning-gw, voice-gw, `iptv-gw` and fulfillment at service-management time.
- A derived `servicesSupported` array (subset of DATA, VOICE, IPTV_MULTICAST) computed from `networkPath` at save time — answers "what can this HomePass physically carry" without requiring consumers to walk the path. Cloud / OTT services delivered over the customer's DATA connection (mobile data via partner-telco SIM, OTT IPTV apps, streaming subscriptions) do NOT appear here — they're handled at the Service-def level (PLM-CFG-01) with `serviceManagementKey = null` or pointing to data.
- **Optional GIS integration via internal geo-client sub-module** (see "GIS integration" section). Address autocomplete on the create-HomePass form, forward/reverse geocoding, polygon containment lookups, map-tile rendering. Strictly additive — every HomePass field a deployment uses today still works without GIS configured. The `geo-client` ships with a Google Maps Platform adapter (Wananchi's current GIS); the adapter pattern lets other providers (ArcGIS, GeoServer, PostGIS-local, Nominatim) swap in without changes elsewhere in the module. Per-HomePass `google_place_id` column captures the stable Place ID when sourced from autocomplete; null otherwise. If a second module (Equipment, WO, Construction) later needs GIS, `geo-client` will be promoted to a foundation gateway at that point — keeping it inside `homepass` for v1 avoids premature abstraction.

RLM-CFG-01 NOT in scope (handled in later DDs):

- The end-to-end **construction request workflow** (Commercial submits → Technical approves → Contractor assigned → MDU formalities → technical design → BH creation) → **HOMEPASS-OP-01**
- **GIS operations on the stored polygons.** v1 stores GeoJSON as opaque JSONB; v1 does NOT perform polygon-vs-point queries, automatic tech_region inference from a HomePass's lat/lng, polygon validation (no self-intersection, no overlaps, no holes), or any map-rendering tooling. These are deferred to a future GIS-focused DD. The schema is GIS-ready; the operations are not. If polygon-vs-point queries become a hot operational need, a future DD migrates from JSONB to PostGIS geometry types.
- The **coverage check API used at acquisition** (one HomePass for one given customer address) → operationalized by **SIP-03** (subscription onboarding), which calls a homepass REST endpoint defined here
- **Field audits** of HomePass data (compare BH ↔ design map) → **FA-01/02/03**
- **WO routing** based on TechRegion → handled by field-ops (work-order management is a separate module)
- **MDU platform** for owner Right-of-Entry, design files, certification artefacts → these are external systems referenced by HOMEPASS-OP-01, not part of this catalog
- **Last-mile customer-side binding** (ONT serial number captured at install, port allocation on a splitter, voice subscriber line on a voipswitch) for "fiber-on-install" regions → captured at install time by **fulfillment / FUL-08** (Equipment binding to AAA/NMS). The HomePass network_path stores the *upstream* nodes (the ones known before install); fulfillment enriches with the last-mile binding when the customer is installed.

Downstream modules (onboarding for coverage check, field-ops for WO routing, fulfillment for activation, billing for per-franchise pricing) consume HomePass / Franchise / TechRegion catalog data via Kafka events.

Module placement. This DD introduces the new homepass module. Franchise + TechRegion + HomePass are all owned by it. Per the HLD module map, homepass lives in cluster A (Config & Catalog) — low write rate, mostly reads from many other modules.

HomePass status codes

HomePass status is **NOT a hardcoded enum**. It is a per-deployment configurable catalog (homepass_status_code — see data model) where admins seed whatever codes their operation needs. Each catalog entry carries **semantic flags** read by downstream modules:

- `is_initial` — valid as the default status when a HomePass is created
- `is_sellable` — sales-eligibility lookups return this HomePass to onboarding/coverage-check
- `is_active` — a customer is installed and consuming service (drives search dual-status, billing trigger)
- `is_terminal` — no further transitions allowed
- `requires_approval_to_enter` — the reference strict BPMN gates transitions INTO this status on a user-task approval
- `triggers_lead_notification` — first time a HomePass enters a status with this flag set, the HomePassReachedSellable event fires (drives lead-notify; default catalog has this on the RFS row)
- `blocks_soft_delete` — HomePass cannot be soft-deleted while in this status

The **transitions between codes** are owned by the deployment's Camunda workflow (see Status Workflow section). The catalog defines which codes EXIST and what each one MEANS; the workflow defines how a HomePass moves between them.

Recommended default seed (for new deployments)

The default catalog shipped with the module — a sane starting point any deployment can extend:

Code	Description	is_initial	is_sellable	is_active	is_terminal	requires_approval_to_enter	triggers_lead_notification	blocks_soft_delete
NPL	Not Planned — address known, no construction committed	✓						
NSN	Not Serviceable Network — planned/in-progress, not yet reachable							

Code	Description	is_initial	is_sellable	is_active	is_terminal	requires_approval_to_enter	triggers_lead_notification	blocks_soft_delete
RFS	Ready For Sales — built, certified, sellable		✓			✓	✓	
WAI	Wired not Active — customer signed up, install in progress		✓			✓		✓
ACT	Active — customer installed and consuming service			✓		✓		✓
RETIRED	Retired — no longer in use (terminal)				✓	✓		

Wananchi KE seed (richer set, mirrors BH vocabulary)

Wananchi's KE deployment seeds the catalog with BH's actual codes (observed in production: NSR Not Ready, NSV No RoE, PH1/PH2/PH3 HFC construction phases, RFC ready for sale construction, RFN ready for sale none-sellable, plus the default-seed codes). The legacy BH code is preserved on each migrated row in a `legacy_status_code` field for audit. New transitions are defined in the deployment's Camunda workflow.

Status semantics are read, never hardcoded

The eligible-lookup endpoint, the soft-delete check, the first-time-sellable event, and the reference BPMN all consult the catalog flags rather than hardcoded code names. A deployment can call its sellable status RFS, LIVE, READY, or anything else — the system behaves the same because downstream code reads `is_sellable`, never the literal code.

2. Business rules

Franchise rules

ID	Rule	Triggered on
R-RLM-CFG-01-F-1	Franchise code must be unique within the deployment (e.g., "002" for Nairobi).	Create, Update
R-RLM-CFG-01-F-2	Franchise code is immutable once the Franchise leaves DRAFT state.	Update
R-RLM-CFG-01-F-3	A Franchise cannot be RETIRED while it has at least one active HomePass pointing at it (status not RETIRED). Migrate or retire the HomePasses first.	Retire
R-RLM-CFG-01-F-4	A Franchise cannot be soft-deleted unless it is RETIRED and has no HomePasses (even retired ones — history must remain attached to a stable Franchise reference).	Delete
R-RLM-CFG-01-F-5	If boundaryGeoJson is present, it must be valid GeoJSON of type Polygon or MultiPolygon. (Validation is structural — well-formed JSON shape — not semantic; v1 does NOT check for self-intersection, hole correctness, or overlaps with other Franchises.)	Create, Update

TechContractorSkill rules

ID	Rule	Triggered on
R-RLM-CFG-01-S-1	TechContractorSkill code must be unique within the deployment. Codes are short uppercase identifiers (e.g., INSTALLATION, MAINTENANCE, SUPPORT, B2B).	Create, Update
R-RLM-CFG-01-S-2	TechContractorSkill code is immutable once any TechContractor references it. (Renaming a skill code would silently change the meaning of every TechContractor using it.)	Update

ID	Rule	Triggered on
R-RLM-CFG-01-S-3	A TechContractorSkill cannot be soft-deleted while any TechContractor references it. The referenced TechContractors must drop the skill from their skill list first.	Delete

TechContractor rules

ID	Rule	Triggered on
R-RLM-CFG-01-C-1	TechContractor code must be unique within the deployment.	Create, Update
R-RLM-CFG-01-C-2	TechContractor code is immutable once the TechContractor leaves DRAFT state.	Update
R-RLM-CFG-01-C-3	TechContractor skills must be a non-empty list of TechContractorSkill codes that all exist in the skill catalog and are not soft-deleted. The same TechContractor may carry multiple skills (e.g., [INSTALLATION, MAINTENANCE]).	Create, Update
R-RLM-CFG-01-C-4	A TechContractor cannot be RETIRED while it is assigned to any TechRegion that has at least one HomePass with status RFS, WAI, or ACT. The contractor must first be unassigned from such TechRegions, or those HomePasses migrated.	Retire
R-RLM-CFG-01-C-5	A TechContractor cannot be soft-deleted unless it is RETIRED and not assigned to any TechRegion (tech_region_contractor join must be empty for this contractor).	Delete

NetworkNode rules

The NetworkNode catalog mirrors the BH dropdown convention: every Head End, Hub, Node, etc. is a controlled entry that admins maintain, not free text. A HomePass's network_path references these by code. Each node may declare a parent — this captures the "from Node TK7 you know the Hub, the Head End, the OLT" convention as a walkable chain.

ID	Rule	Triggered on
R-RLM-CFG-01-N-1	NetworkNode code must be unique within the deployment. Codes follow per-deployment convention (e.g., RNA, TK7, OLT-NRB-WTL-01); the system treats them as opaque identifiers.	Create, Update
R-RLM-CFG-01-N-2	NetworkNode code is immutable once the node leaves DRAFT state. (Renaming a node code would silently break every HomePass referencing it.)	Update
R-RLM-CFG-01-N-3	NetworkNode type must be from the enum (OLT, SPLITTER, FAT, FDT, ONT, HEADEND, DISTRIBUTION_NODE, AMPLIFIER, LINE_EXTENDER, VOIPSWITCH, NMS, OTHER).	Create, Update
R-RLM-CFG-01-N-4	When parent_node_code is set, it must reference an existing NetworkNode in the same deployment. No cycles allowed (the parent chain must terminate). The DD does NOT prescribe which types can parent which — admins seed sensibly per deployment (typical: ONT → FAT → SPLITTER → OLT).	Create, Update
R-RLM-CFG-01-N-5	A NetworkNode cannot be RETIRED while any HomePass with status RFS, WAI, or ACT references it in its network_path.	Retire
R-RLM-CFG-01-N-6	A NetworkNode cannot be soft-deleted unless RETIRED and not referenced by ANY HomePass (even retired ones — historical refs must remain stable).	Delete

HomePassStatusCode rules

Per-deployment configurable catalog of HomePass status codes. The default seed (NPL/NSN/RFS/WAI/ACT/RETIRED) is recommended but every deployment may add, modify (within constraints), or extend.

ID	Rule	Triggered on
R-RLM-CFG-01-HS-1	code must be unique within the deployment. Codes are short uppercase identifiers (e.g., RFS, PH1, NSV).	Create, Update
R-RLM-CFG-01-HS-2	code is immutable once any HomePass row references it.	Update
R-RLM-CFG-01-HS-3	Exactly one catalog entry may have <code>is_initial = true</code> per deployment. (HomePass creation defaults to this code.)	Create, Update
R-RLM-CFG-01-HS-4	An entry cannot be soft-deleted while referenced by any HomePass. Retire instead; retired entries remain queryable for historical rows.	Delete
R-RLM-CFG-01-HS-5	If <code>is_terminal = true</code> , the entry cannot also be <code>is_sellable</code> or <code>is_active</code> (a terminal status is end-of-life, not operational).	Create, Update

HouseType rules

Per-deployment configurable catalog classifying buildings by Multi-Dwelling-Unit class (e.g., M2M = MDU C1 Medium, M1H = MDU AB High, S1H = Single Family). HomePass references one entry by code.

ID	Rule	Triggered on
R-RLM-CFG-01-HT-1	code must be unique within the deployment.	Create, Update
R-RLM-CFG-01-HT-2	code is immutable once any HomePass references it.	Update
R-RLM-CFG-01-HT-3	An entry cannot be soft-deleted while referenced by any HomePass.	Delete

TechRegion rules

TechRegion is an **independent registry** — it does NOT belong to a Franchise. A HomePass carries one or more TechRegion refs and one or more Franchise refs independently; the two registries are not linked at the catalog level.

ID	Rule	Triggered on
R-RLM-CFG-01-T-1	TechRegion code must be unique within the deployment (codes do not repeat).	Create, Update
R-RLM-CFG-01-T-2	TechRegion code is immutable once the TechRegion leaves DRAFT state.	Update
R-RLM-CFG-01-T-3	A TechRegion cannot be RETIRED while it has at least one HomePass in an <code>is_active</code> or <code>blocks_soft_delete</code> catalog status pointing at it (default seed: WAI, ACT). Active HomePasses must drop the reference to this TechRegion first, or be themselves RETIRED.	Retire
R-RLM-CFG-01-T-4	A TechRegion cannot be soft-deleted unless it is RETIRED and no HomePass references it (even retired ones — historical references must remain stable).	Delete
R-RLM-CFG-01-T-5	If <code>boundaryGeoJson</code> is present, it must be valid GeoJSON of type Polygon or MultiPolygon. (Validation is structural — well-formed JSON shape — not semantic; v1 does NOT check for self-intersection, hole correctness, or overlaps with other TechRegions.)	Create, Update
R-RLM-CFG-01-T-7	A TechRegion must have at least one active TechContractor assignment to it (via the <code>tech_region_contractor</code> join, with non-empty <code>skills[]</code>) to be activated. A TechRegion with no contractors cannot install or support any HomePass and is operationally meaningless. The set of skills available in the TechRegion is the union of the <code>skills[]</code> arrays across its active assignments — there is no <code>skills</code> column on <code>tech_region</code> itself.	Activate (DRAFT → ACTIVE)

TechRegion-Contractor assignment rules

ID	Rule	Triggered on
R-RLM-CFG-01-A-1	The assignment's skills array must be non-empty and must be a subset of the contractor's current skills array . (A contractor without MAINTENANCE in their global skill list cannot be assigned to a region for MAINTENANCE.) Validated at assignment-create and assignment-update; also re-validated when a contractor REMOVES a skill (any assignment carrying the removed skill must be updated or deleted first — service-layer check).	Create, Update assignment; Remove skill from contractor
R-RLM-CFG-01-A-2	When a contractor ADDS a skill to their global skills list, existing assignments are NOT auto-extended to include the new skill. Admins decide per region whether the contractor handles the new skill there.	(No automatic propagation — manual admin action)
R-RLM-CFG-01-A-3	An assignment cannot be deleted (or have skills removed leaving the assignment empty) while the resulting state would leave the parent TechRegion without any active assignment covering a skill that ACTIVE HomePasses depend on for service. Soft-rule: warn rather than block in v1; will be hardened when the WO DD lands.	Delete assignment / Update assignment skills

HomePass rules

ID	Rule	Triggered on
R-RLM-CFG-01-H-1	A HomePass unique key is the tuple (country, region, region_l1, region_l2, city, area, sub_area_1, sub_area_2, road_name, building_number, building_name, apartment_number) — deployment-wide. NULL components are treated as the empty string for uniqueness purposes. The system blocks creation of a duplicate. The full tuple (rather than just road + apartment) is used because real-world street names repeat across regions; relying on coarser keys would yield spurious collisions across administrative divisions.	Create
R-RLM-CFG-01-H-2a	HomePass franchiseRefs must be a non-empty list. Every referenced Franchise must exist and be ACTIVE. (A HomePass can be linked to multiple Franchises — e.g., commercial overlap zone, or distinct B2B and B2C franchises covering the same address.)	Create, Update
R-RLM-CFG-01-H-2b	HomePass techRegionRefs must be a non-empty list. Every referenced TechRegion must exist and be ACTIVE. The collective skill coverage across the linked TechRegions should cover the HomePass's operational needs (typically at minimum INSTALLATION and SUPPORT) — but this is NOT a hard rule in v1; the system warns rather than blocks when a HomePass has TechRegions whose union of skills is missing a major skill.	Create, Update
R-RLM-CFG-01-H-3	HomePass accessTechnology is one of GPON, HFC, COPPER, FTTH, WIRELESS (extensible — current KE deployment uses only GPON and HFC; other values reserved).	Create, Update
R-RLM-CFG-01-H-4	HomePass status_code must be a code from the homepass_status_code catalog (deployment-configurable; see data model). The catalog entry must be ACTIVE. The set of valid transitions is NOT defined in this DD or in DRL — it is owned by the deployment's configured Camunda workflow (see Status Workflow section). Status semantic flags (is_sellable, is_active, etc.) are NOT denormalized onto the HomePass row — they're read from the catalog at query time via the foreign key join.	Status set

ID	Rule	Triggered on
R-RLM-CFG-01-H-5	Maker-checker on transitions into statuses with <code>requires_approval_to_enter = true</code> is enforced inside the Camunda workflow: the user task's assignment listener rejects assignment to the proposer. The strict reference BPMN includes this guard; the passthrough reference BPMN does not (no user task). There is no DRL rule under this ID — the constraint is workflow-level.	(workflow)
R-RLM-CFG-01-H-6	On a HomePass's first transition into any status with catalog flag <code>is_sellable = true</code> AND <code>triggers_lead_notification = true</code> (default seed: RFS), the system emits a <code>HomePassReachedSellable</code> event so downstream consumers (notably the lead-capture system / onboarding module) can notify queued leads. Re-transitions back to a sellable status (after <code>sellable → non-sellable → sellable</code>) do NOT re-emit. Enforced in the service layer on every status change emitted by the workflow, gated by the <code>has_been_sellable</code> latch on the HomePass row — workflow-independent, deployment-independent.	Status transition into any <code>is_sellable + triggers_lead_notification</code> status
R-RLM-CFG-01-H-7	A HomePass cannot be soft-deleted unless it is in a status with <code>is_terminal = true</code> (typically RETIRED) AND the catalog entry does NOT have <code>blocks_soft_delete = true</code> AND there are no active customer services bound to this HomePass (cross-module check against fulfillment).	Delete
R-RLM-CFG-01-H-8	Once a HomePass's current status has <code>is_active = true</code> OR <code>blocks_soft_delete = true</code> (typically the WAI/ACT codes in the default seed), <code>franchiseRefs</code> and <code>techRegionRefs</code> become append-only : new refs may be ADDED, but existing refs may NOT be REMOVED. Existing refs may be referenced by active services, billing, or contracts (e.g., a customer's invoice may carry the Franchise that priced their service); silently removing that reference would break history. To remove a ref, the active services must first be terminated (FUL-07) or migrated.	Update
R-RLM-CFG-01-H-9	HomePass <code>activeTerminationPoints</code> is a non-negative integer representing the number of signal-provisioned outlets actually wired at this address (the basis for contractor payment per active termination point certified — from workshop). May be zero (a HomePass can exist without active terminations yet).	Create, Update
R-RLM-CFG-01-H-10	HomePass <code>networkPath</code> is required and describes the chain of network nodes the HomePass sits behind. Structure: an ordered array of { <code>type</code> , <code>code</code> , <code>role</code> , <code>port</code> } objects (see data model). At least one node must exist. <code>captureMode</code> is one of <code>PRE_INSTALLATION</code> (the full path with ports is known at HomePass time — typical for KE fiber-pre-deployed regions) or <code>AT_INSTALLATION</code> (only the upstream path is known at HomePass time; the port and last-mile binding are captured by fulfillment when the customer is installed — typical for regions where the last mile is laid only when a customer is acquired, e.g., TZ where field ops captures port details at install). Each node's <code>type</code> must be from the enum (OLT, SPLITTER, FAT, FDT, ONT, HEADEND, DISTRIBUTION_NODE, AMPLIFIER, LINE_EXTENDER, VOIPSWITCH, NMS, OTHER). Each node's <code>role</code> must be one of <code>service_management</code> (a node the provisioning / voice / monitoring gateways need to talk to), <code>passive</code> (a hop, no service-management traffic), or <code>termination</code> (the last node before the customer endpoint). Each node MAY carry a <code>port</code> string (nullable; format is per-node-type and per-deployment, e.g., 1/2/3 for an OLT, P05 for a splitter, A4-7 for a FAT, null for an ONT where the serial IS the identifier). Ports may be filled later for <code>AT_INSTALLATION</code> mode via the <code>network-path</code> append endpoint.	Create, Update

ID	Rule	Triggered on
R-RLM-CFG-01-H-13	When a node's port is present, it must be a non-empty string of at most 64 characters. There is no semantic format validation in v1 (every deployment has its own port-naming convention) — the field is treated as an opaque identifier. Future per-deployment regex validation can be added without changing this rule's intent.	Create, Update, Network-path append
R-RLM-CFG-01-H-14	Address fields are mutable post-create : country, region, region_l1, region_l2, city, area, sub_area_1, sub_area_2, road_name, building_number, building_name, apartment_number, floor, building_num_floors, building_num_apartments, latitude, longitude, altitude, map_code, map_link may be corrected at any point in the HomePass lifecycle, including after WAI/ACT. Every change is audit-logged and emits a HomePassAddressCorrected event. Address corrections often originate from WO notes at install time (in BH today, field crews flag address problems in WO free-text notes; in V3, the WO module — separate DD — calls a structured PATCH on the HomePass). The uniqueness check (R-RLM-CFG-01-H-1) is re-evaluated on every address change.	Update
R-RLM-CFG-01-H-15	Every networkPath.nodes[i].code must reference an existing ACTIVE entry in the network_node catalog (see data model). The catalog entry's type must match the node's type. Catalog-driven references replace BH's free-text node fields and let admins maintain a controlled list of OLTs, FATs, splitters, etc. — matching today's BH dropdown convention (Node TK7 → FRT 12-47 etc.).	Create, Update, Network-path append
R-RLM-CFG-01-H-16	RoE fields are coherent: if roe_signed_date is non-null, the HomePass's status MUST NOT be in a code with not_serviceable_reason = "NO_ROE" semantics (i.e., the deployment-configured RoE-pending status, default NSV). Conversely, if not_serviceable_reason = "NO_ROE", then roe_signed_date MUST be null. The system warns rather than blocks in v1 (the RoE DD may have legitimate edge cases — e.g., RoE signed but document not yet uploaded, or RoE signed but voided by re-survey); will harden when the RoE DD lands. roe_document_link is free-form URL with no v1 validation beyond URL-shape check.	Create, Update
R-RLM-CFG-01-H-17	GIS coherence warning (advisory, NOT blocking). When BOTH coordinates (latitude + longitude) AND admin-hierarchy fields (country, region_l1, city, etc.) are present, AND the deployment has GIS configured, the create/update path SHOULD reverse-geocode the coordinates and compare the resulting admin hierarchy against the typed fields. Significant divergence (e.g., GPS in Mombasa but typed regionL1 = Nairobi County) raises a warning visible to the admin; admin may accept (override) or correct. Never blocks — geocoding can fail, admin may have legitimate reason for divergence (e.g., a building straddling an admin boundary), and RLM-CFG-01 works without GIS at all.	Create, Update
R-RLM-CFG-01-H-18	google_place_id (and any future GIS-provider place identifier) is immutable once non-null on a HomePass. An admin must NOT rebind a HomePass to a different Place ID after the initial assignment — that would silently change the physical reference. The proper way to "fix" a wrong Place ID is to retire the HomePass and create a new one. If the Place ID is null on create, it MAY be filled later (e.g., by a backfill batch job calling enrich-from-geo); once set, it stays set.	Update

ID	Rule	Triggered on
R-RLM-CFG-01-H-11	HomePass serviceManagementEndpoints is a derived map { serviceFamily → {nodeCode, port} } computed from networkPath at save time. It names, per service family (default set: data, voice, iptv_multicast, monitoring; extensible per deployment), which specific node code AND port the corresponding gateway should address. If networkPath contains no node with role service_management for a given service family, that family's key is absent from the map. The port part of each endpoint may be null if the resolved node has no port captured (typical for AT_INSTALLATION HomePasses before install runs). Multiple keys may legitimately resolve to the same {nodeCode, port} (e.g., data and iptv_multicast both via the same OLT port).	Create, Update
R-RLM-CFG-01-H-12	HomePass servicesSupported is a derived array (subset of DATA, VOICE, IPTV_MULTICAST) computed from networkPath at save time. A node-type → service-family mapping (configurable per deployment, default: OLT → DATA plus OLT → IPTV_MULTICAST if the OLT is configured for multicast video, VOIPSWITCH → VOICE, dedicated video HEADEND → IPTV_MULTICAST) is applied across the path; the union of supported families is stored. A phone-only HomePass (POTS over copper, or VOIPSWITCH-only) yields servicesSupported = ["VOICE"]. A typical Wananchi fiber HomePass with both a data OLT and a voice route yields ["DATA", "VOICE"]. IPTV_MULTICAST specifically means "the physical termination supports multicast IPTV requiring HomePass-level OLT configuration" (the kind that needs an STB); cloud/OTT IPTV apps that run over the customer's DATA connection are NOT modeled here — they're Service-level concerns on the Service def (PLM-CFG-01). Used by onboarding to filter package eligibility at acquisition and by cvm for service-mix segmentation.	Create, Update

Catalog lifecycle states (Franchise, TechRegion, TechContractor)

Same model as PLM-CFG-04, simpler:

- **DRAFT** — admin still building it; not visible to other modules
- **ACTIVE** — published; referenced by HomePasses (or by TechRegion ↔ TechContractor join), downstream modules see it
- **RETIRED** — terminal; cannot be re-activated

Transitions: DRAFT → ACTIVE (admin activates after rules pass), ACTIVE → RETIRED (admin retires, terminal). No "INACTIVE" middle state for Franchise / TechContractor / TechRegion — these are not time-bound entities; they exist or they don't.

TechContractorSkill uses the same lifecycle (DRAFT / ACTIVE / RETIRED) for consistency, though in practice skill entries rarely retire.

3. Module owner and impacted modules

Role	Module	What it does in this process
Owner	homepass (NEW MODULE — created by this DD)	Owns Franchise, TechContractorSkill, TechContractor, TechRegion, HomePass catalog data, REST APIs, the homepass admin UI, the DRL rules KJAR (homepass-rules — also new), and the Camunda BPMN definitions for HomePass status transitions (reference workflows shipped, deployment-selected)
Impacted (cross-references at enablement)	homepass (TechContractor validates skills; TechRegion validates that ≥1 active TechContractor is assigned before activation; HomePass validates Franchise + TechRegion refs)	Self-referential — homepass module verifies its own internal references; no other module is called during a save
Impacted (cross-module check on HomePass delete)	fulfillment (active services bound to a HomePass)	R-RLM-CFG-01-H-7 — homepass calls fulfillment REST to check for active customer services before allowing HomePass soft-delete

Role	Module	What it does in this process
Impacted (downstream consumers)	onboarding (SIP-03 — coverage check at acquisition; receives HomePassReachedSellable for lead-notification), fulfillment (FUL-03 — service activation reads HomePass for endpoints, and may call the eligible-contractors lookup for installation crew selection), field-ops / WO module (separate DD — reads HomePass and the eligible-contractors lookup; owns all WO routing logic, slot/capacity scheduling, and multi-hop role transfers — see "Cross-DD coordination" below), billing (per-franchise pricing at invoice time reads Franchise), cvm (segmentation by Franchise / TechRegion), search index, BSS Backoffice	Consume FranchiseCreated/..., TechContractorSkillCreated/..., TechContractorCreated/..., TechRegionCreated/..., NetworkNodeCreated/..., HomePassCreated/Updated/AddressCorrected/StatusChanged/ReachedSellable/Retired Kafka events
Coordination	onboarding (lead-capture sub-area, future ILM-CFG / SIP-03)	R-RLM-CFG-01-H-6 — when a HomePass first reaches a sellable status, the emitted HomePassReachedSellable event triggers the lead-capture system to notify previously-queued leads. The notification delivery itself goes through notification-gw (called by the consumer in onboarding, not by homepass directly)
Gateways	—	None used directly. Downstream consumers may call notification-gw to deliver the lead-notification
Engines	Drools KIE (homepass-rules KJAR), Camunda (HomePass status workflow)	Drools: rule evaluation on every Franchise / TechContractor / TechRegion / HomePass create / update / delete. Camunda: every status-transition request is a process instance of homepass-status-transition (the BPMN definition is deployment-configured — see "Status workflow").

The HomePass module embeds Camunda as its workflow engine for status transitions (see "Status workflow"). The full construction workflow (HOMEPASS-OP-01) ALSO uses Camunda but is a separate DD — it produces HomePass status changes through the same status-transition workflow contract.

4. Foundation references

DB → FOUNDATION_DB.md · Drools → FOUNDATION_DROOLS.md · Kafka → FOUNDATION_KAFKA.md · Cache → FOUNDATION_CACHE.md · Auth → FOUNDATION_AUTH.md.

Technical design

Module placement

Layer	Where
Frontend	Shared BSS Backoffice UI (Network section, Franchise / TechRegion / HomePass sub-areas)
Backend	New homepass module (created by this DD), sub-areas franchise-catalog/, tech-contractor-skill-catalog/, tech-contractor-catalog/, tech-region-catalog/, network-node-catalog/, homepass-status-code-catalog/, house-type-catalog/, homepass-catalog/, plus optional geo-client/ for GIS integration
Rules	New Rules KJAR homepass-rules (Java/Maven project, regardless of homepass module stack)
Events	Internal Event Bus (Kafka), topic families sophix.homepass.franchise.*, sophix.homepass.techregion.*, sophix.homepass.homepass.*

Single-tenant: same codebase per country, country-specific config loaded at startup.

homepass module — ecosystem (RLM-CFG-01 view)

```

flowchart LR
    UI[BSS Backoffice UI Network] -->|REST| HP[homepass module]
    BULK[Bulk import API] -->|REST| HP
    HP -->|invoke| RE[Rules Engine homepass-rules]
    HP -->|publish| EB[Kafka]
    EB --> ONB[onboarding SIP-03 coverage and lead-notify]

```

```

EB --> FUL[fulfillment FUL-03 cache]
EB --> FO[field-ops TechRegion routing]
EB --> BIL[billing Franchise pricing cache]
EB --> CVM[cvm cache]
EB --> SEARCH[search index]
HP -.cross-module check on delete.-> FULREST[fulfillment REST active services]
HP -.optional, internal geo-client.-> GIS[Google Maps Platform]

```

homepass calls out cross-module exactly once: when soft-deleting a HomePass, to confirm no active services are bound to it. The dotted line to Google Maps is the optional GIS path through the internal `geo-client` sub-area — not a foundation gateway, not cross-module. Otherwise homepass is a self-contained catalog.

Codebase

This DD creates the homepass module. Stack choice (Java/Spring Boot OR Laravel) is independent of other modules.

Java/Spring Boot layout

```

modules/homepass/                                ← NEW MODULE
  pom.xml
  src/main/java/com/sophix/homepass/
    franchise-catalog/
      domain/
        Franchise.java
        FranchiseStatus.java
      repository/
        FranchiseRepository.java
      service/
        FranchiseService.java
        FranchiseValidationService.java          ← invokes Drools
    web/rest/
      FranchiseResource.java
      dto/{CreateFranchiseRequest, UpdateFranchiseRequest, FranchiseResponse}.java
    event/
      FranchiseEventPublisher.java
    exception/
      FranchiseException.java
  tech-region-catalog/
    domain/
      TechRegion.java
      TechRegionStatus.java
    repository/
      TechRegionRepository.java
    service/
      TechRegionService.java
      TechRegionValidationService.java
    web/rest/
      TechRegionResource.java
      dto/...
    event/
      TechRegionEventPublisher.java
  homepass-catalog/
    domain/
      HomePass.java
      HomePassStatus.java
      AccessTechnology.java
      NetworkPath.java
      NetworkPathNode.java
      ServiceManagementEndpoints.java
      StatusTransition.java
      StatusTransitionApproval.java
    repository/
      HomePassRepository.java
      StatusTransitionRepository.java
    service/
      HomePassService.java
      HomePassValidationService.java
      StatusTransitionService.java              ← maker-checker logic
      BulkImportService.java
    web/rest/
      HomePassResource.java
      BulkImportResource.java
      dto/...
    event/
      HomePassEventPublisher.java
  geo-client/                                     ← optional GIS adapter (Step 20)
    adapter/
      GeoAdapter.java                           ← interface
      GoogleMapsAdapter.java                   ← v1 default
      NoOpAdapter.java                         ← when homepass.gis.enabled=false
    domain/

```

```

    GeocodeResult.java
    ReverseGeocodeResult.java
    AutocompleteSuggestion.java
    PlaceDetails.java
    ContainmentResult.java
    BiasContext.java
service/
    GeoClient.java ← façade used by HomePassService and BulkImportService
    GeoLookupCache.java ← reads/writes geo_lookup_cache
web/rest/
    GeoEnrichmentResource.java ← POST /enrich-from-geo, GET /map
config/
    GeoClientProperties.java ← maps homepass.gis.* config keys
shared/
messaging/ ← outbox/inbox workers
src/main/resources/
config/liquibase/changelog/homepass/
    001_franchise.xml
    002_tech_contractor_skill.xml
    003_tech_contractor.xml
    004_tech_region.xml
    005_tech_region_contractor.xml
    006_network_node.xml
    007_homepass_status_code.xml
    008_house_type.xml
    009_homepass.xml
    010_homepass_franchise.xml
    011_homepass_tech_region.xml
    012_audit_log.xml
    013_geo_lookup_cache.xml
workflows/
    homepass-status-transition--strict.bpmn ← Camunda BPMN (deployed at module startup)
    homepass-status-transition--passthrough.bpmn ← reference strict maker-checker (KE default)
    homepass-status-transition--passthrough.admin-driven.bpmn ← reference passthrough admin-driven

```

Laravel layout (if homepass is Laravel)

```

modules/homepass/ ← NEW MODULE
composer.json
app/
    Domain/Franchise/
        Models/Franchise.php
        Services/FranchiseService.php
        Http/Controllers/FranchiseController.php
        Http/Requests/{Create,Update}FranchiseRequest.php
        Events/{FranchiseCreated,Updated,Retired}.php
    Domain/TechRegion/
        Models/TechRegion.php
    ...
    Domain/HomePass/
        Models/{HomePass, HomePassStatusTransition}.php
        Services/{HomePassService, StatusTransitionService, BulkImportService}.php
        Http/Controllers/{HomePassController, BulkImportController}.php
    ...
    Domain/GeoClient/
        Contracts/GeoAdapter.php ← optional GIS adapter (Step 20)
        Adapters/{GoogleMapsAdapter, NoOpAdapter}.php ← interface
        Models/{GeocodeResult, ReverseGeocodeResult, AutocompleteSuggestion, PlaceDetails, ContainmentResult}.php
        Services/{GeoClient, GeoLookupCache}.php
        Http/Controllers/GeoEnrichmentController.php ← POST /enrich-from-geo, GET /map
    Domain/Shared/
    Domain/Messaging/ ← outbox/inbox workers
database/migrations/
    2026_05_12_000001_create_franchise_table.php
    2026_05_12_000002_create_tech_contractor_skill_table.php
    2026_05_12_000003_create_tech_contractor_table.php
    2026_05_12_000004_create_tech_region_table.php
    2026_05_12_000005_create_tech_region_contractor_table.php
    2026_05_12_000006_create_network_node_table.php
    2026_05_12_000007_create_homepass_status_code_table.php
    2026_05_12_000008_create_house_type_table.php
    2026_05_12_000009_create_homepass_table.php
    2026_05_12_000010_create_homepass_franchise_table.php
    2026_05_12_000011_create_homepass_tech_region_table.php
    2026_05_12_000012_create_audit_log_table.php
    2026_05_12_000013_create_geo_lookup_cache_table.php
resources/workflows/
    homepass-status-transition--strict.bpmn ← Camunda BPMN (deployed at module startup)
    homepass-status-transition--passthrough.bpmn ← reference strict maker-checker (KE default)
    homepass-status-transition--passthrough.admin-driven.bpmn ← reference passthrough admin-driven

```

DRL files live in the separate Java/Maven KJAR project modules/homepass-rules/ (see FOUNDATION_DR00LS.md).
Laravel homepass calls KIE Server over HTTPS REST.

Foundation-specific configurations

Foundation	Configuration for RLM-CFG-01
DB	New homepass database in cluster A (Config & Catalog). Tables franchise, tech_contractor_skill, tech_contractor, tech_region, tech_region_contractor, network_node, homepass_status_code, house_type, homepass, homepass_franchise, homepass_tech_region, audit_log in schema homepass_catalog; optional geo_lookup_cache when GIS is configured. Messaging tables in shared messaging schema. Transition state lives in Camunda (separate engine schema), not in this module.
Drools	New KJAR artifact: homepass-rules- <code>{country}_{version}</code> .jar. PLM-CFG-04 contributes DRL files under rules/franchise-catalog/, rules/tech-region-catalog/, rules/homepass-catalog/.
Kafka	New topic families: sophix.homepass.franchise.*, sophix.homepass.techregion.*, sophix.homepass.homepass.*. Key format: homepass:franchise:{id}, homepass:techregion:{id}, homepass:homepass:{id}.
Cache	Redis cache for HomePass coverage reads (very high read rate from onboarding at acquisition time): sophix:homepass:homepass:{id} 24h TTL; reverse-lookup cache for address → homepass id with shorter TTL.
Auth	Roles: HOMEPASS_ADMIN (full CRUD on Franchise/TechRegion + HomePass create/update + propose status transitions), HOMEPASS_APPROVER (approve maker-checker transitions to RFS+), HOMEPASS_VIEWER (read-only), HOMEPASS_BULK_IMPORTER (bulk import API access). JWT verified locally by homepass.
Camunda	New process key: homepass-status-transition. BPMN files in src/main/resources/workflows/. KE ships homepass-status-transition--strict.bpmn (maker-checker on requires_approval_to_enter = true codes). Deployments choosing the admin-driven flow ship homepass-status-transition--passthrough.bpmn instead. Both files declare the same process key, the same end events (transition-approved / transition-rejected), and the same start-variable shape. Module startup posts every BPMN in the workflows folder to Camunda's /deployment/create REST endpoint. End-event execution listener implementation: StatusTransitionEndListener class registered with Camunda. Candidate group HOMEPASS_APPROVER resolves at KE to the network-operations team. History retention P5Y on this process (long retention because it's the transition audit ledger).
GIS (optional)	Config keys read at module boot: homepass.gis.enabled (default false), homepass.gis.adapter (default google_maps), homepass.gis.google_maps.api_key (from secret store — Vault / k8s Secret / env, never logged), homepass.gis.cache.enabled (default true when GIS enabled), homepass.gis.cache.ttl.{geocode, reverse_geocode, autocomplete, place_details} with sensible defaults. When enabled = false (or when the adapter is noop), the geo-client façade returns empty results for every call and the integration points (/enrich-from-geo, /map, autocomplete in the UI) silently degrade. The API key has its own quota at the Google side; the cache layer keeps cost predictable.

Frontend

BSS Backoffice UI, Network section. Three sub-areas, each with list / detail / edit screens.

Screen	Purpose
Franchise list / detail / create-edit	CRUD on Franchise; lifecycle Activate / Retire
TechRegion list / detail / create-edit	CRUD on TechRegion (filtered by Franchise); lifecycle Activate / Retire
HomePass list / detail / create-edit	CRUD on HomePass; status transitions with maker-checker UI showing proposed transitions awaiting approval
HomePass bulk import	File upload (CSV matching the BH NS2 schema or new clean schema); validation report; commit
HomePass map view (out of scope here)	Map overlay showing HomePasses by status — deferred to a future UX-focused DD

Maker-checker UI for HomePass status transitions:

- A user with HOMEPASS_ADMIN proposes a transition (e.g., NSN → RFS). The proposal is created with status PROPOSED, the proposer recorded.

- A user with HOMEPASS_APPROVER (must be a different user than the proposer) sees a queue of pending proposals, opens each, and approves or rejects.
- On approve: the HomePass status flips, an event is emitted, the proposal record is marked APPROVED.
- On reject: the proposal record is marked REJECTED with a reason; the HomePass status does not change.

Data model

franchise

Column	Type	Notes		
id	TEXT PK	ULID		
code	TEXT UNIQUE NOT NULL	e.g. "002"		
name	TEXT NOT NULL	e.g. "Nairobi"		
description	TEXT NULL			
boundary_geojson	JSONB NULL	optional GeoJSON Polygon or MultiPolygon representing the Franchise's geographic footprint. Stored as opaque JSONB; v1 does no spatial operations on it (per scope). Future DD may migrate to PostGIS geometry.		
status	TEXT NOT NULL	DRAFT \	ACTIVE \	RETIRED
created_at	TIMESTAMP NOT NULL			
created_by	TEXT NOT NULL			
updated_at	TIMESTAMP NOT NULL			
updated_by	TEXT NOT NULL			
retired_at	TIMESTAMP NULL	set when status → RETIRED		
deleted_at	TIMESTAMP NULL	soft-delete		
has_been_active	BOOLEAN NOT NULL DEFAULT false	latches true on first activation		

tech_contractor_skill

Small reference catalog. Admins add new skills via REST + UI; downstream consumers (FUL-03 installation routing, ASR-01 ticket routing) read by code. Renamed in this iteration from the earlier tech_region_skill — skills belong to contractors, not regions.

Column	Type	Notes		
id	TEXT PK	ULID		
code	TEXT UNIQUE NOT NULL	short uppercase identifier (e.g., INSTALLATION, SUPPORT)		
name	TEXT NOT NULL	human-friendly name		
description	TEXT NULL			
status	TEXT NOT NULL	DRAFT \	ACTIVE \	RETIRED
created_at / updated_at / created_by / updated_by		standard audit cols		
retired_at / deleted_at		standard soft-delete		

Initial seed for KE deployment (and a reasonable default elsewhere): INSTALLATION, MAINTENANCE, SUPPORT. More can be added (B2B, EMERGENCY, etc.) without code changes.

tech_contractor

The contractor company or team performing field work for one or more TechRegions. Carries the skills.

Column	Type	Notes		
id	TEXT PK	ULID		

Column	Type	Notes		
code	TEXT UNIQUE NOT NULL	short identifier (e.g., ACME_FIBER, HELPDESK_NRB)		
name	TEXT NOT NULL	full company / team name		
description	TEXT NULL			
skills	TEXT[] NOT NULL	non-empty array of tech_contractor_skill.code values; per R-RLM-CFG-01-C-3. Postgres array type — supports WHERE 'INSTALLATION' = ANY(skills) queries for downstream routing.		
status	TEXT NOT NULL	DRAFT \	ACTIVE \	RETIRED
created_at / updated_at / etc.		standard audit cols		
has_been_active	BOOLEAN NOT NULL DEFAULT false			
retired_at / deleted_at		standard soft-delete		

Indexes:

- idx_contractor_code UNIQUE on (code)
- idx_contractor_skills_gin GIN on (skills) — supports skill-filtered routing

Forward-looking: a future EM-02 (Contractor management) DD may extend this table with contract details, SLA terms, payment terms, etc. For RLM-CFG-01 v1, code + name + skills + lifecycle is enough.

tech_region

Column	Type	Notes		
id	TEXT PK	ULID		
code	TEXT UNIQUE NOT NULL	unique within the deployment (NOT scoped to a Franchise)		
name	TEXT NOT NULL			
description	TEXT NULL			
boundary_geojson	JSONB NULL	optional GeoJSON Polygon / MultiPolygon — the TechRegion's operational footprint. The polygon represents both administrative scope and network coverage; the two collapse in practice (a TechRegion's operational territory equals where the network reaches). If a deployment ever needs a regulatory-vs-operational distinction, a separate column can be added later.		
status	TEXT NOT NULL	DRAFT \	ACTIVE \	RETIRED
created_at / updated_at / etc.		standard audit cols		
has_been_active	BOOLEAN NOT NULL DEFAULT false			
retired_at / deleted_at		standard soft-delete		

Indexes: idx_techregion_code UNIQUE on (code).

There is NO foreign key from tech_region to franchise. The two registries are independent (see scope section). There is NO skills column on tech_region — skills belong to contractors; the available skills in a TechRegion are the union of its assigned TechContractors' skills.

tech_region_contractor

Join table: which TechContractors serve which TechRegion, **for which skills**. Many-to-many with per-assignment skill scope.

A contractor's assignment to a region carries its own subset of the contractor's global skills. The same contractor may install in one region but only maintain in another — the assignment encodes this scope. Routing queries filter on the **join's** `skills[]`, not the contractor's global skill list.

Column	Type	Notes
tech_region_id	TEXT NOT NULL	FK tech_region.id
tech_contractor_id	TEXT NOT NULL	FK tech_contractor.id
skills	TEXT[] NOT NULL	non-empty subset of the contractor's <code>skills</code> array — the skills this contractor handles IN THIS TechRegion (R-RLM-CFG-01-A-1)
created_at	TIMESTAMP NOT NULL	when the assignment was made
created_by	TEXT NOT NULL	actor

Primary key: (tech_region_id, tech_contractor_id). Indexes:

- `idx_trc_contractor` on (tech_contractor_id) — "which TechRegions does this contractor serve?"
- `idx_trc_skills_gin` GIN on (skills) — supports "find contractors in this region with skill X"

Routing example (FUL-03 / WO DD wants an installation contractor for HomePass hp_X):

1. `homepass_tech_region` → list of TechRegions for `hp_X`
2. `tech_region_contractor` → rows where `tech_region_id IN (those_regions) AND 'INSTALLATION' = ANY(skills)` — note this filters on the **join's** skills, NOT the contractor's full skill list
3. Resolve to TechContractor records; filter by `status = 'ACTIVE'`

This correctly returns "contractors who do INSTALLATION in THIS region", not "all contractors in this region who happen to have INSTALLATION in their global skill list."

network_node

Reference catalog of every node a HomePass's `network_path` may reference: OLTs, FATs, splitters, headends, distribution nodes, voipswitches, etc. Admin-managed CRUD, per deployment. Replaces BH's free-text Head End / Hub Number / Node fields with a controlled list (matching today's BH dropdown convention).

Column	Type	Notes											
id	TEXT PK	ULID											
code	TEXT UNIQUE NOT NULL	per-deployment convention (e.g., RNA, TK7, OLT-NRB-WTL-01); referenced by HomePass.network_path.nodes[i].code											
type	TEXT NOT NULL	OLT \	SPLITTER \	FAT \	FDT \	ONT \	HEADEND \	DISTRIBUTION_NODE \	AMPLIFIER \	LINE_EXTENSION \	VOIPSWITCH \	NMS \	OTHER

Column	Type	Notes											
name	TEXT NOT NULL	descriptive name (e.g., RUNDA OLT, FRT 12-47) — matches BH's "code + descripti on" drop down shape											
parent _node_ code	TEXT NULL	optional parent in the topology chain (e.g., a FAT's parent is its Splitter; the Splitter's parent is the OLT). W alkable from leaf to root. v1 makes no semantic claim about which type can parent which — admins seed sensibly.											
descri ption	TEXT NULL	free-text notes											
metada ta	JSONB NULL	nullable extensio n point for vendor, model, GPS, co nfigurati on para meters etc. — a future ne twork-to pology DD may populate this; v1 leaves it empty.											
status	TEXT NOT NULL	DRAFT \	ACTIVE \	RETIRE D									
create d_at / u pdated _at / etc.		standard audit cols											

Column	Type	Notes											
has_been_active	BOOLEAN NOT NULL DEFAULT false												
retired_at / deleted_at		standard soft-delete											

Indexes:

- idx_networknode_code UNIQUE on (code) — primary lookup
- idx_networknode_type on (type) — filter by node type (e.g., "list all OLTs")
- idx_networknode_parent on (parent_node_code) — supports "all children of X" queries for topology rendering

Forward-looking note: a future **network-topology DD** may (a) auto-populate this catalog from OLT/FAT auto-discovery feeds (like Wananchi's `malogs.wananchi.com/gpon_auto_find` portal that polls OLTs every 5 minutes for discovered ONTs), (b) reconcile discovered ONT serials against HomePass.network_path terminal nodes, and (c) flag drift. Out of scope for RLM-CFG-01 v1 — the catalog is admin-managed here.

homepass_status_code

Per-deployment configurable catalog of HomePass status codes with their semantic flags. The default seed (NPL/NSN/RFS/WAI/ACT/RETIRED) is recommended but every deployment may add codes; Wananchi KE seeds with BH's richer set (NSR, NSV, PH1/2/3, RFC, RFN, plus defaults). See Section 1 glossary.

Column	Type	Notes		
id	TEXT PK	ULID		
code	TEXT UNIQUE NOT NULL	short uppercase identifier (e.g., RFS, NSV, PH1)		
description	TEXT NOT NULL	human-readable label		
display_order	INTEGER NOT NULL DEFAULT 0	for sorting in UI dropdowns		
is_initial	BOOLEAN NOT NULL DEFAULT false	exactly one entry true per deployment (R-RLM-CFG-01-HS-3) — drives HomePass creation default		
is_sellable	BOOLEAN NOT NULL DEFAULT false	sales-eligibility flag; eligibility lookup returns this HomePass		
is_active	BOOLEAN NOT NULL DEFAULT false	customer-installed flag; search returns bound subscription		
is_terminal	BOOLEAN NOT NULL DEFAULT false	no further transitions; mutex with is_sellable and is_active (R-RLM-CFG-01-HS-5)		
requires_approval_to_enter	BOOLEAN NOT NULL DEFAULT false	reference strict BPMN gates transitions INTO this code on a user task		
triggers_lead_notification	BOOLEAN NOT NULL DEFAULT false	first time HomePass enters a code with this flag, HomePass ReachedSellable fires (R-RLM-CFG-01-H-6)		
blocks_soft_delete	BOOLEAN NOT NULL DEFAULT false	service-layer DELETE check refuses while HomePass is in this status		
status	TEXT NOT NULL	DRAFT \	ACTIVE \	RETIRED (the catalog row's own lifecycle, separate from the HomePass status it describes)
created_at / updated_at / etc.		standard audit cols		
retired_at / deleted_at		standard soft-delete		

Indexes:

- idx_status_code_code UNIQUE on (code)
- idx_status_code_initial partial on (is_initial) WHERE is_initial = true — supports R-RLM-CFG-01-HS-3 lookup
- idx_status_code_flags on (is_sellable, is_active, is_terminal) — supports reporting queries by semantic category

house_type

Per-deployment configurable catalog of building density / dwelling-class codes (e.g., M2M = MDU C1 Medium, M1H = MDU AB High, S1H = Single-family). Used for segmentation, sales prioritization, construction sizing.

Column	Type	Notes		
id	TEXT PK	ULID		
code	TEXT UNIQUE NOT NULL	short identifier (e.g., M2M, S1H)		
description	TEXT NOT NULL	full label (e.g., MDU C1 Medium)		
display_order	INTEGER NOT NULL DEFAULT 0	UI ordering		
status	TEXT NOT NULL	DRAFT \	ACTIVE \	RETIRED
created_at / updated_at / etc.		standard audit cols		
retired_at / deleted_at		standard soft-delete		

Indexes: idx_house_type_code UNIQUE on (code).

homepass

Column	Type	Notes				
id	TEXT PK	ULID				
access_technology	TEXT NOT NULL	GPON \	HFC \	COPPER \	FTTH \	WIRELESS — the physical reach to the address
network_path	JSONB NOT NULL	ordered chain of network nodes the HomePass sits behind, plus captureMode. See structure below.				
service_management_endpoints	JSONB NOT NULL DEFAULT '{}'	derived from network_path at save time: map of service type → node code. See structure below.				
services_supported	TEXT[] NOT NULL DEFAULT '{}'	derived from network_path at save time: array of service-family codes (subset of DATA, VOICE, IPTV_MULTICAST). Per R-RLM-CFG-01-H-12.				

Column	Type	Notes				
Address hierarchy fields		The flat address_line of BH is replaced by a structured hierarchy. All fields except country and road_name are free-text strings, all nullable. Per-deployment, admins use whichever levels their geography actually needs (a deployment may use only country + region_l1 + city; another may use all 8 admin levels). The full tuple is the uniqueness key — see R-RLM-CFG-01-H-1.				
country	TEXT NOT NULL	ISO 3166-1 alpha-2 code (e.g., KE, TZ, UG, SN)				
region	TEXT NULL	broadest admin or geographic grouping (e.g., Coast, East Africa). Nullable.				
region_l1	TEXT NULL	top admin division (e.g., Nairobi County in KE, Dakar Region in SN).				
region_l2	TEXT NULL	second-level admin division (e.g., Westlands Sub-County).				
city	TEXT NULL	city or town (e.g., Nairobi).				
area	TEXT NULL	neighbourhood / locality (BH "Area" field, e.g., Kilimani).				
sub_area_1	TEXT NULL	finer subdivision within area.				
sub_area_2	TEXT NULL	even finer subdivision.				
road_name	TEXT NOT NULL	street/road (BH "Road Name", e.g., Wood Avenue)				
building_number	TEXT NULL	building/plot number on the road (BH "Building #", e.g., 12)				
building_name	TEXT NULL	building name (BH "Building Name", e.g., Finewood Apartments)				
apartment_number	TEXT NULL	unit identifier within the building (BH "Apt", e.g., M18). NULL for single-family homes.				

Column	Type	Notes				
floor	TEXT NULL	e.g. M18, BH "Floor" field — kept as free-text (per-building schemes vary too much for a global catalog)				
building_num_floors	INTEGER NULL	total floors in the building (denormalized from the design map's blue building box, e.g., 13). Same value across all HomePass rows in the same building — admins fill once and clone.				
building_num_apartments	INTEGER NULL	total apartments in the building (e.g., 48). Same across rows in the same building. Reporting/capacity use.				
property_type	TEXT NOT NULL	RES \	COM \	MIXED \	OTHER \	TST (test) — small enum (rarely changes); not promoted to a catalog. OTHER covers buildings that are neither residential nor commercial in the standard sense — stadiums, churches, mosques, schools, hospitals, government buildings — where the HomePass exists but the standard RES / COM segmentation doesn't fit.
owner_occupied	BOOLEAN NOT NULL DEFAULT false					
house_type_code	TEXT NULL	FK to house_type.code (configurable catalog) — replaces free-text house_type. Examples: M2M (MDU C1 Medium), S1H (Single family), etc.				
outlets	INTEGER NOT NULL DEFAULT 0	physical outlets at this address				
active_termination_points	INTEGER NOT NULL DEFAULT 0	signal-provisioned outlets — counts FAT terminations for fiber, TAP for HFC. The deployment-wide aggregate (sum across HomePasses) is the "active termination points in the network" KPI. R-RLM-CFG-01-H-9 governs increments.				

Column	Type	Notes				
latitude	NUMERIC(10,7) NULL	point location				
longitude	NUMERIC(10,7) NULL	point location				
altitude	NUMERIC(8,2) NULL	optional altitude in meters				
map_code	TEXT NULL	optional legacy/external map atlas reference (BH "Map Code"); free-text per-deployment				
map_link	TEXT NULL	optional URL to design-map artifact (PDF). Document foundation deferred — see HLD §18.				
status_code	TEXT NOT NULL	FK to homepass_status_code (configurable catalog) — see R-RLM-CFG-01-H-4 and Section 1 glossary. Semantic flags (is_sellable, is_active, is_terminal, blocks_soft_delete, requires_approval_to_enter, triggers_lead_notification) are read from the catalog row at query time , NOT denormalized onto the HomePass row. The catalog has ~10–15 rows and lives in PG buffer cache, making the join effectively a memory lookup; the idx_homepass_status_code index supports it efficiently. Flag changes on a catalog row take effect immediately on the next read with no background re-denorm job.				
has_been_sellable	BOOLEAN NOT NULL DEFAULT false	latches true on the first transition into any status with is_sellable = true; used by R-RLM-CFG-01-H-6 to fire HomePassReachedSellable exactly once.				
legacy_status_code	TEXT NULL	preserves the original BH status code at migration (e.g., PH1, NSV, RFC) when it differs from the V3 canonical code. Audit / reporting only.				

Column	Type	Notes				
not_serviceable_reason	TEXT NULL	optional JSONB-ish reason code when the HomePass is in a non-sellable status (e.g., NO_ROE, OWNER_REFUSED, INFRA_PENDING). Helps workflows pick the right sub-flow.				
perm_date	DATE NULL	permission date — workshop field				
survey_date	DATE NULL	survey date				
roe_signed_date	DATE NULL	Right-of-Entry signing date — when the building owner signed the RoE agreement granting Wananchi the right to install equipment in the building. Denormalized onto every HomePass in the building (RoE is signed once per building, covers all apartments). Audit / reference only — RoE lifecycle (signing workflow, renewal, expiry) is owned by a future RoE / construction-survey DD.				
roe_document_link	TEXT NULL	URL to the signed RoE document. Free-text URL in v1; will become a structured document reference when the document foundation (HLD §18) ships. Same value across all HomePasses in the same building (admins fill once, clone propagates).				

Column	Type	Notes				
google_place_id	TEXT NULL	Optional — populated when the HomePass was created via Google Places Autocomplete (or backfilled later via geocoding). Google's stable identifier for this building/address. Provides a durable cross-reference into Google's data even if admins later rename building_name or correct typos. Useful for Street View links, deduplication checks ("are these two HomePass rows really the same physical address?"), and future GIS adapter changes. Free-text; the gateway adapter format is opaque to this DD. R-RLM-CF G-01-H-17 keeps it immutable once set. NULL when the HomePass was created manually without GIS enrichment — that path remains fully supported.				
directions	TEXT NULL	how to physically reach the address (free-text, used by field crews)				
comments	TEXT NULL	general comments				
notes	TEXT NULL	additional notes (BH has these as three separate fields; we preserve the split)				
created_at / updated_at / etc.		standard audit cols				
retired_at / deleted_at		standard soft-delete				

Note: franchise_ref and tech_region_ref are NOT columns on homepass. The HomePass↔Franchise and HomePass↔TechRegion relationships are many-to-many — see the join tables below.

homepass_franchise

Column	Type	Notes
homepass_id	TEXT NOT NULL	FK homepass.id
franchise_ref	TEXT NOT NULL	FK franchise.id
created_at	TIMESTAMP NOT NULL	when the link was established
created_by	TEXT NOT NULL	actor

Primary key: (homepass_id, franchise_ref). Indexes:

- idx_hpfr_franchise on (franchise_ref) — supports "all HomePasses in this Franchise" queries

homepass_tech_region

Column	Type	Notes
homepass_id	TEXT NOT NULL	FK homepass.id
tech_region_ref	TEXT NOT NULL	FK tech_region.id
created_at	TIMESTAMP NOT NULL	
created_by	TEXT NOT NULL	

Primary key: (homepass_id, tech_region_ref). Indexes:

- idx_hptr_techregion on (tech_region_ref) — supports "all HomePasses serviced by this TechRegion" queries (used by ticket / WO routing)

Structure of network_path

The network_path JSONB column captures the chain of network nodes the HomePass sits behind, **including the port on each node when known**. Modeling as JSONB (not a normalized homepass_network_node table) is a deliberate v1 choice: the path is small (typically 3–5 nodes), heterogeneous (different node sequences per technology), and most reads want the whole path or a single endpoint — patterns JSONB serves cleanly. If reporting needs like "all HomePasses behind OLT X port 1/2/3" become hot and frequent, a future DD can derive a normalized table from this column or materialize a view; the JSONB stays as the source of truth.

```
{
  "captureMode": "PRE_INSTALLATION",
  "nodes": [
    { "type": "OLT",      "code": "OLT-NRB-WTL-01",    "role": "service_management", "port": "1/2/3" },
    { "type": "SPLITTER", "code": "SPL-NRB-WTL-FAT-12", "role": "passive",           "port": "P05"   },
    { "type": "FAT",      "code": "FAT-NRB-WTL-12-A4", "role": "termination",       "port": "A4-7"  },
    { "type": "ONT",      "code": "ONT-SN-1234567890", "role": "termination",       "port": null    }
  ]
}
```

Fields:

- captureMode — PRE_INSTALLATION (full path with ports known at HomePass time, typical for KE) or AT_INSTALLATION (only the upstream chunk is known at HomePass time; port and last-mile binding completed by fulfillment when the customer is installed, typical for TZ-style fiber-on-customer-install regions where field ops captures port details on the install ticket)
- nodes[] — ordered chain. Order matters: index 0 is upstream-most (closest to the network core), the last entry is closest to the customer.
- nodes[i].type — enum: OLT, SPLITTER, FAT, FDT, ONT, HEADEND, DISTRIBUTION_NODE, AMPLIFIER, LINE_EXTENDER, VOIPSWITCH, NMS, OTHER. Extensible — adding a new node type means an enum addition and (if needed) a service-management mapping rule update.
- nodes[i].code — vendor-or-internal identifier for this specific node (e.g., the OLT shelf identifier, the FAT plate code). The string format is per-deployment.
- nodes[i].role — service_management (a node that the gateways need to call for service operations — typically OLT and VOIPSWITCH and NMS), passive (just a hop — splitters, FATs, amplifiers in a pass-through role), or termination (the last node before the customer endpoint).
- nodes[i].port — nullable string identifier of the specific port / slot / line on this node that serves this HomePass (e.g., "1/2/3" for an OLT port, "P05" for a splitter port, "A4-7" for a FAT plate position). null when the node has no meaningful port concept (e.g., an ONT where the serial number IS the identifier) or when the port is not yet known (AT_INSTALLATION mode, before install). Format is opaque, per-deployment, validated only for non-empty when present (R-RLM-CFG-01-H-13).

For AT_INSTALLATION captureMode, the path captured at HomePass time covers only the upstream nodes (and their ports may be filled with placeholder or null values for the downstream entries). When the customer is installed (FUL-08), fulfillment appends and/or updates the last-mile nodes — adding the specific ONT serial, filling in the OLT port assignment, etc. — via a dedicated PATCH endpoint. The HomePass's network_path is then UPDATED; the original captureMode = AT_INSTALLATION is preserved for audit / reporting (so we can tell at-install regions from pre-install ones) even when the actual path is now complete.

Structure of service_management_endpoints

Derived from network_path at save time. Pre-computed so provisioning-gw, voice-gw, iptv-gw, fulfillment, and any other gateway can read one short JSONB blob instead of walking the path. Each entry carries both the node code AND the port — both are required for the gateway to address a specific subscriber line.

The keys name a **service family** (the kind of service being managed), NOT the gateway that handles it. Symmetric naming across families: every key answers "what service family is this endpoint for?"

```
{
  "data":      { "nodeCode": "OLT-NRB-WTL-01", "port": "1/2/3" },
  "voice":     { "nodeCode": "VOIPSWITCH-NRB-01", "port": "EXT-5421" },
  "iptv_multicast": { "nodeCode": "OLT-NRB-WTL-01", "port": "1/2/3" },
  "monitoring": { "nodeCode": "NMS-MAIN", "port": null }
}
```

The keys are service-family identifiers — data, voice, iptv_multicast, monitoring for now; extensible (e.g., a future deployment that offers a managed-CCTV service over the HomePass network would add cctv). The values are objects {nodeCode, port} pulled from network_path.nodes[] where role = "service_management" and the node's type matches the service family by a deployment-configurable mapping (default: OLT → data + iptv_multicast, VOIPSWITCH → voice, NMS → monitoring). The port is copied verbatim from the matched node — it may be null if the matched node's port hasn't been captured yet (AT_INSTALLATION path before install). If no node in network_path matches a given service family, that family is absent from the map (rather than null) — keeps the JSON small.

Note that data and iptv_multicast may legitimately point at the same node + port (e.g., an OLT delivering both subscriber-line data service and multicast IPTV through the same GPON port). Different services, same physical endpoint.

This map is recomputed on every HomePass save (cheap — walk a 3–5-element array) and stored. Consumers always read from here, never re-derive.

How fulfillment uses service_management_endpoints to dispatch commands

At activation (FUL-03) and on every subsequent service operation (suspend, upgrade, terminate — FUL-04, FUL-05), fulfillment resolves the target node via the following flow.

Inputs at the moment fulfillment runs:

- A subscription_id, which carries the customer_id, the package_ref (SIP-01), and the homepass_id (HomePass at the installed address).
- The operation type (ACTIVATE, SUSPEND, UPGRADE, TERMINATE).

Fulfillment then:

1. **Loads the Package** (SIP-01). The Package lists its constituent service_refs[].
2. **For each Service in the Package, loads the Service def** (PLM-CFG-01). The Service def carries:
 - networkProfileShape.provisionerKey — which Provisioner adapter to use (e.g., GPON_RES_PROFILE, VOIP_SIP_PROFILE)
 - networkProfileShape.params — the recipe inputs (downstream/upstream Mbps, profile template, etc.)
 - equipmentRequirement — what equipment the service expects (e.g., GPON_ONT)
 - serviceManagementKey — **which endpoint family on the HomePass this Service consumes** (e.g., "data" for an internet Service, "voice" for a VoIP Service, "iptv_multicast" for a multicast STB-IPTV Service). Defined by **R-PLM-CFG-01-6. Nullable** — see below.
3. **Loads the HomePass** (this DD). The HomePass carries service_management_endpoints, the pre-computed map of {serviceFamily → {nodeCode, port}}.
4. **Resolves the node and port:** for each Service, looks up homepass.service_management_endpoints[service.serviceManagementKey]. That returns both the nodeCode and the port to address. If port is null (AT_INSTALLATION mode, install not yet run), fulfillment rejects the operation with a clear error — the install workflow must complete first.
5. **Calls the Provisioner adapter** (registered by provisionerKey per PLM-CFG-01's activation flow): Provisioner.activate(params, customer, equipment, targetNodeCode, targetPort). The Provisioner translates the recipe into concrete NMSGateway / AAAGateway / VoiceGateway calls, addressing the resolved node code AND port.

When serviceManagementKey is null (partner-telco fulfilled services — e.g., a bundled mobile SIM data pack from a partner MVNO, an OTT cloud-IPTV app, a streaming subscription resold over the customer's data connection): the Service is NOT addressed at the HomePass. The Provisioner adapter sources its target from elsewhere — typically the Service def's partnerEndpoint metadata (the partner's API URL, account ref, etc.). Fulfillment skips the HomePass endpoint lookup for these. PLM-CFG-01 documents the null-key case; this DD's responsibility is just to NOT require an endpoint when the Service didn't ask for one.

Worked example for a customer activating a Package containing two Services (an Internet 100MB and a VoIP Line):

```

subscription_id → package_ref pkg_INET_VOIP_RES, homepass_id hp_01HX

Package pkg_INET_VOIP_RES → service_refs = [svc_INET_100M, svc_VOIP_LINE]

Service svc_INET_100M → {
  networkProfileShape: { provisionerKey: "GPON_RES_PROFILE",
    params: { downstreamMbps: 100, ... } },
  equipmentRequirement: "GPON_ONT",
  serviceManagementKey: "data"
}

Service svc_VOIP_LINE → {
  networkProfileShape: { provisionerKey: "VOIP_SIP_PROFILE",
    params: { codec: "G711", ... } },
  equipmentRequirement: "ATA",
  serviceManagementKey: "voice"
}

HomePass hp_01HX → service_management_endpoints = {
  "data": { nodeCode: "OLT-NRB-WTL-01", port: "1/2/3" },
  "voice": { nodeCode: "VOIPSWITCH-NRB-01", port: "EXT-5421" }
}

Fulfillment dispatches:
→ GponResProvisioner.activate(params, customer, ont,
  "OLT-NRB-WTL-01", "1/2/3")
  → which in turn issues NMSGateway commands addressing OLT-NRB-WTL-01 port 1/2/3
→ VoipSipProvisioner.activate(params, customer, ata,
  "VOIPSWITCH-NRB-01", "EXT-5421")
  → which in turn issues VoiceGateway commands addressing VOIPSWITCH-NRB-01 extension EXT-5421

```

Two clean lookups; no business logic re-derives the path; the gateways do not know anything about HomePasses, Packages, or Services — they just take a node code, port, and profile, and act.

Worked example for a Package that ALSO bundles a partner-telco mobile-data SIM (Wananchi + Safaricom co-branded plan):

```

Package pkg_FIBER_PLUS_MOBILE → service_refs = [svc_INET_100M, svc_MOBILE_5GB]

Service svc_MOBILE_5GB → {
  networkProfileShape: { provisionerKey: "PARTNER_TELCO_MOBILE",
    params: { dataAllowanceGb: 5, partnerCode: "SAF" } },
  equipmentRequirement: "SIM_CARD",
  serviceManagementKey: null ← key intentionally absent
}

Fulfillment for svc_MOBILE_5GB:
→ does NOT call homepass.service_management_endpoints (key is null)
→ calls PartnerTelcoMobileProvisioner.activate(params, customer, sim)
  → which uses params.partnerCode to pick the partner API and provisions there

```

The HomePass plays no role in the mobile-data activation. Same Package, two Services, two different fulfillment paths.

How is this different from what's already in the Service definition?

The Service def (PLM-CFG-01) and the HomePass's service_management_endpoints answer **different questions** and must not be confused:

Question	Answered by	Example value
What is this offering?	Service def (PLM-CFG-01)	"Internet 100MB" — recurring, addressable, equipment GPON_ONT, tax group VAS
How do I provision it (which adapter, what params)?	Service def (PLM-CFG-01) — networkProfileShape.provisionerKey + params	GPON_RES_PROFILE + { downstreamMbps: 100, ... }
What kind of endpoint does it need (which slot on the HomePass)?	Service def (PLM-CFG-01) — serviceManagementKey (R-PLM-CFG-01-6)	data (or voice, iptv_multicast, or null for partner-telco-fulfilled services)
Where can the offering be sold?	Package (SIP-01) — targetFranchises, targetTechRegions	KE Nairobi + Mombasa, GPON tech regions
Which specific node should the command go to for THIS installed customer?	HomePass service_management_endpoints	OLT-NRB-WTL-01
What's the full network path behind the customer's address?	HomePass network_path	[OLT, splitter, FAT, ONT] chain

Roles in this separation:

- **Service def is portable.** "Internet 100MB" is the same Service whether the customer is in Nairobi or Mombasa. It carries the abstract instructions (the Provisioner key, the params, the equipment requirement, the endpoint slot kind). It does NOT know about specific OLTs.
- **HomePass is address-specific.** It knows the customer's physical location and the network topology serving that location — which OLT, which voipswitch, which video headend. It does NOT know what's being sold to the customer.
- **Fulfillment is the engine that combines them.** It reads the Service def to know what command to send, and the HomePass to know where to send it.

This separation keeps the Service catalog small (one Service per offering, not one per offering-per-location) and confines topology changes (new OLT comes online, old OLT decommissioned, HomePass re-homed) to HomePass rows — no Service def touched, no Package version bumped.

Cross-DD coherence with PLM-CFG-01

This flow depends on `Service.serviceManagementKey` (PLM-CFG-01 R-6). The two DDs are coherent: PLM-CFG-01 defines the column, Fact, DRL rule, and event payload; RLM-CFG-01 defines the endpoint family keys. The default value for an addressable Service that omits the key is "data". For services not addressed at the HomePass (partner-telco mobile SIM data, OTT IPTV apps, third-party streaming bundles), `serviceManagementKey` is null — fulfillment skips the HomePass endpoint lookup and the Provisioner adapter sources its target from elsewhere on the Service def.

Indexes:

- `idx_homepass_address` UNIQUE on (country, COALESCE(region, ""), COALESCE(region_l1, ""), COALESCE(region_l2, ""), COALESCE(city, ""), COALESCE(area, ""), COALESCE(sub_area_1, ""), COALESCE(sub_area_2, ""), road_name, COALESCE(building_number, ""), COALESCE(building_name, ""), COALESCE(apartment_number, "")) — implements R-RLM-CFG-01-H-1, deployment-wide.
- `idx_homepass_country_region` on (country, region_l1, region_l2) — supports admin filters and "all HomePasses in this region" queries
- `idx_homepass_city_area` on (city, area) — common geographic filter
- `idx_homepass_road_building` on (road_name, building_name) — supports partial-text searches like BH's address search dialog
- `idx_homepass_status_code` on (status_code) — supports reporting by status AND the JOIN to `homepass_status_code` for the eligibility/active/terminal lookups (the JOIN-based pattern; the catalog table sits in PG buffer cache, so the join cost is negligible at v1 scale)
- (Eligibility lookup query shape: `SELECT h.* FROM homepass h JOIN homepass_status_code c ON c.code = h.status_code WHERE c.is_sellable = true AND ...franchise filters...` The join is bounded by the ~10-row catalog; planner uses a hash join with the catalog hashed first, near-zero overhead per HomePass row.)
- `idx_homepass_data_node` on ((service_management_endpoints->>'data')) — supports "all HomePasses behind OLT X" queries on the most-hit endpoint
- `idx_homepass_network_path_nodes_gin` GIN on (network_path->'nodes') — supports arbitrary "find all HomePasses with this node code anywhere in their path" queries
- `idx_homepass_services_supported` GIN on (services_supported) — supports filtering by service type
- `idx_homepass_house_type` on (house_type_code) — supports house-type segmentation
- spatial index `idx_homepass_geo` on (latitude, longitude) when present — future map view

Address uniqueness in v1 is **deployment-wide** rather than per-franchise. Reasoning: a physical apartment is one physical apartment regardless of which commercial franchises overlap on it. The full address tuple (country down to apartment) is the uniqueness key — coarser keys would yield spurious collisions where street names repeat across regions.

Status transitions — owned by Camunda, no DB table

There is **no** `homepass_status_transition` **table** in this module. Camunda is the single source of truth for transition state:

- `historyService.createHistoricProcessInstanceQuery().processDefinitionKey("homepass-status-transition").variableValueEquals("homepassId", X)` returns all transition attempts for a HomePass — proposed, in-flight, completed, rejected.
- `taskService.createTaskQuery().processDefinitionKey("homepass-status-transition").candidateGroup("HOMEPASS_APPROVER")` returns the approver queue.

- Process variables carry homepassId, fromStatusCode, toStatusCode, toStatusCatalogEntry (the catalog row with its flags), proposedBy, proposedReason, transitionStartedAt.
- The end event reached (transition-approved / transition-rejected) is the outcome. User-task completion variables carry decidedBy, decidedAt, decidedReason.

If a deployment needs a curated transitions ledger for BI / compliance reporting, the pattern is a **read-side projection** built from Camunda history events (consumer of the engine's event stream), not a transaction-time table in this DD.

API endpoints (GET /api/homepasses/{id}/transitions, GET /api/homepasses/transitions/pending, POST .../approve, POST .../reject) all delegate to Camunda's history and task services directly — see API section.

geo_lookup_cache

Local cache of GIS adapter responses. Avoids billing-per-call duplicates against Google Maps and absorbs latency for repeat lookups. Populated by GeoLookupCache.put(), read by GeoLookupCache.get() from inside the geo-client sub-area. Only present when GIS is configured; harmless empty table otherwise.

Column	Type	Notes				
cache_key	TEXT PK	composite of operation + normalized input — e.g., geocode:wood-avenue-12-kilimani-nairobi-ke, reverse:-1.21476,36.85234, place-details:ChIJ...				
operation	TEXT NOT NULL	geocode \	reverse-geocode \	autocomplete \	place-details \	contains — for stats and selective invalidation
payload	JSONB NOT NULL	the adapter's normalized response, stored as-is				
provider	TEXT NOT NULL	which adapter produced this (google_maps, arcgis, etc.) — invalidate on adapter swap				
fetched_at	TIMESTAMP NOT NULL	when the underlying call was made				
expires_at	TIMESTAMP NOT NULL	TTL: geocoding 30 days, reverse-geocoding 30 days, autocomplete 1 hour, place-details 7 days, contains invalidated on boundary_geojson PATCH				
hit_count	INTEGER NOT NULL DEFAULT 0	incremented on each cache hit — feeds adapter cost-reporting				

Indexes:

- idx_geo_cache_expires on (expires_at) — supports the cleanup job that prunes expired rows nightly
- idx_geo_cache_operation on (operation, fetched_at) — supports per-operation analytics

The cleanup job deletes expired rows nightly; an aggressive cap (e.g., 5M rows) keeps the table bounded even if the cleanup falls behind.

audit_log_entry (shared by all homepass features)

Standard pattern per HLD §13. Three entity types: Franchise, TechRegion, HomePass. Each operation logged.

Messaging tables

In homepass DB, schema messaging. Standard outbox/inbox per FOUNDATION_KAFKA.md.

GIS integration (optional, additive — internal to this module)

This section describes how RLM-CFG-01 integrates with a GIS service for address enrichment, geographic lookups, and map rendering. **The integration is strictly opt-in** — a deployment without GIS configured continues to work exactly as before. Every HomePass field can still be entered manually. The GIS integration is value-add, never a hard dependency.

The integration lives inside the homepass module as an internal geo-client sub-area, not as a separate foundation gateway. The decision is deliberate: RLM-CFG-01 is the only consumer today, and the abstraction cost of a separate foundation is not justified by one consumer. If a second module (Equipment for ONT GPS validation, WO for crew dispatch optimization, Construction for design-zone polygons) later needs the same capabilities, geo-client will be promoted to a geo-gw foundation at that point. The interface contract documented below is forward-compatible with that promotion.

The default GIS provider is **Google Maps Platform** (Wananchi's current GIS — formerly known as Google Maps PRO). The adapter pattern in geo-client keeps the door open for other backends: ArcGIS, GeoServer, Carto, a local PostGIS server, or a custom in-house map service. Switching providers is an adapter swap behind the interface; the rest of homepass code does not change.

Two GIS layers

V3 distinguishes two GIS layers:

1. **Public geographic data** — streets, addresses, admin boundaries, coordinates. Sourced from Google Maps Platform via the geo-client adapter.
2. **Wananchi's private network topology** — FAT positions, OLT-to-FAT chains, building-level coverage. Sourced from PDF design maps for human reference and from the network_node catalog for structured queries. See "Design map PDFs and structured topology" below.

The two layers are addressed with different mechanisms in this DD: the public layer through geo-client API calls, the private layer through map_link URL references and the network_node catalog.

geo-client sub-module

An internal sub-area of the homepass module providing the GIS capability. Responsibilities:

- abstracts the GIS provider behind a stable adapter interface (Java interface or PHP contract — same shape in both stacks)
- holds the API key / credentials per deployment (config-driven, never logged)
- caches responses in the geo_lookup_cache table (Google Maps bills per call; same address geocoded twice in 30 days hits cache, not Google)
- normalizes responses to a stable internal shape across adapter implementations

The Java adapter contract:

```
public interface GeoAdapter {
    GeocodeResult geocode(String address, BiasContext bias);
    ReverseGeocodeResult reverseGeocode(double lat, double lng);
    List<AutocompleteSuggestion> autocomplete(String query, String sessionToken, BiasContext bias);
    PlaceDetails placeDetails(String providerPlaceId);
    ContainmentResult contains(double lat, double lng, List<String> layers);
}
```

v1 ships with **one adapter**, GoogleMapsAdapter, calling:

- Geocoding API for geocode and reverseGeocode
- Places API (Place Autocomplete + Place Details) for autocomplete and placeDetails
- In-process polygon-in-point test against franchise.boundary_geojson / tech_region.boundary_geojson for contains (no provider call needed; the polygons live in this module's DB)

The providerPlaceId returned by geocode / reverseGeocode / placeDetails is opaque to the rest of homepass; with the Google adapter it's a Google Place ID, with an ArcGIS adapter it would be an ArcGIS feature ID, with PostGIS it might be a row PK. The module stores it verbatim in homepass.google_place_id and round-trips it.

When a deployment has the homepass.gis.enabled flag false, all geo-client methods are no-ops (returning empty results) and the integration points described below silently degrade — no errors, just no enrichment.

Adding a non-Google adapter later is a matter of writing a new class implementing GeoAdapter and flipping homepass.gis.adapter in config. No other module code changes.

Three integration points in RLM-CFG-01

(A) Address autocomplete on the create-HomePass form (Backoffice UI)

The "New HomePass" wizard's address field becomes a Google Places Autocomplete dropdown (via Google adapter). Admin types "Wood Aven", sees dropdown candidates, picks one. Form fires `place-details` → fills the structured `country / region_l1 / city / area / road_name / building_number / building_name / latitude / longitude / google_place_id` fields. Admin reviews, overrides any field if needed, submits.

This is convenience, not authority. Admin can always:

- Type the address fields manually without ever invoking autocomplete (the form's text inputs remain editable)
- Override any auto-filled field after selection
- Skip autocomplete entirely on deployments where GIS is not configured

The autocomplete dropdown is implemented client-side; RLM-CFG-01 itself does not call `geo-client` for this — the Backoffice UI talks to Google Places directly. RLM-CFG-01 receives the resulting fields in the POST body like any other create request.

(B) Coordinate ↔ admin-hierarchy enrichment at bulk import and post-hoc backfill

Migration CSVs from BH sometimes carry GPS coordinates (parsed from WO notes like `1°12'46.9"S 36°52'35.3"E`) but no admin hierarchy. Or vice versa: hierarchy but no coordinates. `BulkImportService` can detect this and, when GIS is configured, queue a background enrichment job per row:

- Row has `latitude/longitude` but null `country/region_l1/city` → call `reverse-geocode` → fill the admin fields
- Row has admin hierarchy but null coordinates → call `geocode` → fill `latitude/longitude`

The enrichment is best-effort. If the GIS adapter is unreachable or returns no result, the row imports with whatever fields the CSV provided (NULLs preserved). The enrichment is logged on the audit trail with a `HomePassGeoEnriched` event marker.

An explicit endpoint is also exposed:

- POST `/api/homepasses/{id}/enrich-from-geo` — admin-triggered. Runs the enrichment logic on demand. Useful for: backfilling old rows that predate GIS setup; correcting a row where the manually-typed address is suspected wrong; testing a new GIS provider on a sample row before flipping a deployment.

(C) HomePass map view in the Backoffice

A new Backoffice screen displays HomePasses as map pins. The map base layer is a Google Maps embed (JavaScript Maps API, called from the Backoffice client; `geo-client` is not involved at this layer — it's just a UI library call). On top of the base layer:

- HomePass pins, colored by `status_code` (sellable green, active blue, NSV red, NSN yellow, etc. — admins can configure colors per deployment)
- `TechRegion` boundary `geojson` polygons as transparent fills
- `Franchise` boundary `geojson` polygons as transparent fills with thicker borders
- Clustered at low zoom (1000+ pins in view), individual at high zoom
- Click pin → drawer with full HomePass detail; admins with `HOMEPASS_ADMIN` can navigate to edit

RLM-CFG-01 exposes a tile-friendly endpoint:

- GET `/api/homepasses/map?bbox=west,south,east,north&zoom=N&statusFilter=...&franchiseRef=...` → GeoJSON `FeatureCollection` of HomePasses in that bounding box. Returns `[{type:"Feature", geometry:{type:"Point", coordinates:[lng,lat]}, properties:{id, statusCode, accessTechnology, buildingName}}, ...]`. Pagination via `bbox` tiling; the client makes one request per visible tile.

The map view is opt-in and unaffected by lack of GIS. Even without geocoding, HomePasses created with explicit `latitude/longitude` (manually entered or imported with GPS in CSV) appear on the map; HomePasses without coordinates do not (they're still findable via the regular list / search UI). A deployment can run the map view with zero GIS calls if no enrichment is needed.

Design map PDFs and structured topology

Network design today lives in PDF artifacts: pink FAT polygons, labeled FAT codes (104-25, 104-26), blue building boxes with apartment-count annotations (e.g., 48/13). No programmatic GIS contains this data, and automated extraction is

non-trivial (vector vs raster classification, named-layer parsing for vector PDFs, manual digitization for rasters).

V3 handles network design with two coexisting artifacts:

PDFs as link-attached reference. The `map_link` column on `homepass` stores the URL of the building's / area's design PDF. Construction uploads PDFs to a document store (Google Drive, S3, SharePoint — deployment choice); HomePass rows reference the URL. The PDF is opened from the Backoffice for human reference — field crews on tablets, design teams in reviews, admins during corrections. When a document foundation lands (HLD \$18 deferred), `map_link` upgrades from raw URL to structured document reference without breaking the v1 contract.

Structured topology in the `network_node` catalog. Admins transcribe FATs / OLTs / Splitters from PDFs into the `network_node` catalog (code, type, name, `parent_node_code`, optional latitude / longitude / metadata) as construction work creates the HomePasses. The catalog grows incrementally with operational work. Queries against the structured catalog ("which FATs are in this area?", "what's the parent chain of FAT 104-25?") are then possible alongside the visual PDF reference.

Automated PDF-to-GeoJSON conversion is out of V3 scope. If prioritized later, it lands as a separate DD (working name GIS-MIGRATION-01 / NETWORK-TOPOLOGY-01) feeding into the existing `network_node.metadata` and `tech_region.boundary_geojson` columns with no schema change here.

Forward-looking: PostGIS as a future optimization

In v1, polygons on `franchise.boundary_geojson` and `tech_region.boundary_geojson` are stored as plain JSONB. Polygon containment queries (via the `geo-client` adapter) work by fetching all polygons of the relevant layer and testing them in application code (or in the adapter against Google Maps' containment API).

This scales fine for Wananchi's volumes (~6 Franchises + ~50 TechRegions across KE). When polygon counts grow or spatial queries become hotter (e.g., a future construction-planning DD does "find all HomePasses within 100m of this proposed new FAT"), v1 can upgrade to:

- Enable the PostGIS extension on the `homepass` DB: `CREATE EXTENSION postgis;`
- Migrate `boundary_geojson` to `boundary_geom GEOMETRY(MULTIPOLYGON, 4326)` columns
- Add GiST spatial indexes
- Move polygon-in-point tests from in-process to direct in-database queries

The migration is non-breaking (the API surface stays the same; only the storage and execution change). PostGIS becomes attractive when call-volume to Google's containment API gets expensive OR when the query needs are richer than Google's API offers (e.g., "find HomePasses within polygon X but outside polygon Y").

Not v1 work. Documented here so a future reader understands the upgrade path.

Cross-DD coordination

Added in the Cross-DD section below as a new row noting GIS is owned within this module, and a separate row for the future PDF/network-topology DD.

Status workflow

Every HomePass status transition flows through a **Camunda BPMN process instance**. The DD defines the **contract** the workflow must honor, ships **two reference BPMN definitions** (strict maker-checker and passthrough admin-driven), and leaves the choice between them — or any deployment-specific variant — to per-deployment configuration. There are NO transition-validity rules in DRL or in DB constraints; the workflow IS the rules.

Camunda is the engine; maker-checker is a pattern

The two terms are not alternatives:

- **Camunda** is the workflow engine. It runs BPMN processes, persists state, routes work to user tasks, records history, and signals end events. It does not define what happens in a transition — it runs whatever BPMN is deployed under the `homepass-status-transition` process key.
- **Maker-checker** is an approval pattern in which the proposer (maker) and the approver (checker) are different users. It is implemented here as a Camunda user task with an assignment listener that rejects assignment to the proposer.

Every HomePass status transition runs as a Camunda process instance. The strict reference BPMN includes the maker-checker user task; the passthrough reference BPMN does not. Both run on Camunda.

Whether a given transition is gated on approval is data, not code: the `requires_approval_to_enter` flag on the destination status code's catalog row. The strict BPMN reads this flag at its decision gateway; the passthrough BPMN ignores

it (always auto-applies). A deployment changes its approval policy by flipping the catalog flag — no BPMN or application redeployment.

Why every transition routes through Camunda

- **Single code path** in the service layer — `startProcessInstance` regardless of deployment policy.
- **Uniform audit lineage** — every transition is a process instance in Camunda history with the same shape (`proposedBy`, `proposedAt`, `decidedBy`, `decidedAt`, `reasons`, `variables`).
- **HOMEPASS-OP-01 integration** — the future construction workflow is also Camunda and calls the status-transition workflow as a child process via call-activity. A bypass path would break this integration.
- **Policy evolves without a code release** — catalog flag flip, BPMN swap if needed.

Process key and variables

- **Process key**: `homepass-status-transition` (each deployment ships one or more BPMN under this key, selected at runtime via Camunda's process-versioning).
- **Start variables** (passed by the service layer when proposing a transition):
 - `homepassId` (string) — target `HomePass`
 - `fromStatusCode` / `toStatusCode` (string) — proposed transition
 - `toStatusCatalogEntry` (object) — the destination status's catalog row including all semantic flags, snapshotted at start so the workflow gateway has a stable view
 - `proposedBy` (string) — actor
 - `proposedReason` (string, nullable) — comment
- **End event types**:
 - `transition-approved` → service-layer listener updates `homepass.status_code` to `toStatusCode`, sets `has_been_sellable = true` if this is the first time entering an `is_sellable` AND triggers `lead_notification` status, emits `HomePassStatusChanged`, and (if the sellable latch flipped) emits `HomePassReachedSellable`. All persisted atomically with an outbox event in one DB transaction.
 - `transition-rejected` → listener emits `HomePassTransitionRejected`. `HomePass` row unchanged. No DB row in this module — the rejection lineage is in Camunda's history.

Contract that any workflow must honor

A deployment may ship any BPMN under the `homepass-status-transition` key, but the BPMN MUST:

1. **End with exactly one of the two end events** (`transition-approved` or `transition-rejected`). Listeners on these end events do the actual `HomePass` row update + event emission.
2. **Honor maker-checker** if the BPMN has an approval user task: the task's assignee-validation listener must reject assignment to `proposedBy` (same user can't approve their own proposal). Both reference BPMNs include this listener pattern.
3. **Carry the standard process variables** named above, so the history queries (`/api/homepasses/{id}/transitions`) can project them uniformly across deployments.

The BPMN owns: which transitions are valid (any not-defined-in-BPMN transition fails to start the workflow), which require user tasks, which require additional gates (e.g., checks against external systems), what notifications to send during the workflow itself.

BPMN file layout

`homepass-status-transition` BPMNs live in the `homepass` module's workflow folder (per `FOUNDATION_CAMUNDA` convention):

```
modules/homepass/
  ■■■■ src/main/resources/
    ■■■■ workflows/
      ■■■■ homepass-status-transition--strict.bpmn      ← reference strict maker-checker (KE default)
      ■■■■ homepass-status-transition--passthrough.bpmn ← reference passthrough admin-driven
```

Both files declare the **same process key** `homepass-status-transition` and the **same end-event names** (`transition-approved`, `transition-rejected`). A deployment ships exactly ONE of them — Camunda's process definition deployer picks up whichever `.bpmn` is in the workflow folder. KE ships the strict one; deployments that prefer admin-driven flips ship the passthrough one. A custom deployment can replace either with its own BPMN as long as the

contract holds (same process key, same end-event names, same start-variable shape, same listener wiring).

Both reference BPMNs are checked into the homepass module's resources folder and deployed at module startup via Camunda's deployment API (the module's startup code reads everything under `workflows/` and posts to the engine REST `/deployment/create`).

Reference BPMN 1: strict maker-checker (KE default)

```
flowchart LR
    Start((Start)) --> GW{requires_approval_to_enter}
    GW -->|true| UT[User Task HOMEPASS_APPROVER]
    GW -->|false| EndAuto((End transition-approved auto-applied))
    UT -->|APPROVE| EndApproved((End transition-approved))
    UT -->|REJECT| EndRejected((End transition-rejected))
```

BPMN contains:

- Process attributes

- `id="homepass-status-transition"` — process key
- `isExecutable="true"`
- `camunda:historyTimeToLive="P5Y"` — Camunda history kept 5 years (long retention because this is the audit ledger; cluster-level retention may extend further)

- Start event — start-transition

- Receives the 5 process variables passed in by `StatusTransitionService.startTransition()`:
- `homepassId` (String) — target HomePass ULID
- `fromStatusCode` (String) — current status code
- `toStatusCode` (String) — proposed status code
- `toStatusCatalogEntry` (Json) — snapshot of the catalog row including all semantic flags, captured at start so the gateway evaluation is stable across the workflow's lifetime
- `proposedBy` (String) — actor user ID
- `proposedReason` (String, optional) — comment

- Exclusive gateway — gw-needs-approval

- Condition on the YES path: `${toStatusCatalogEntry.requires_approval_to_enter == true}`
- Default flow goes to the auto-applied end event

- User Task — task-approve-transition

- `camunda:candidateGroups="HOMEPASS_APPROVER"`
- `camunda:assignee=""` (deliberately empty — claimed at runtime by an approver from the candidate group)

- **Task-creation listener** (`org.camunda.bpm.engine.delegate.TaskListener` on create): emits a notification through the notification gateway — payload { `channel: "email"`, `to: "<HOMEPASS_APPROVER group inbox>"`, `template: "homepass-transition-pending"`, `variables: { homepassId, fromStatusCode, toStatusCode, proposedBy, proposedReason }` }. The handler endpoint is `homepass-module/api/internal/notify/homepass-transition-pending`.

- **Assignment listener** (assignment event): rejects assignment if `task.assignee == proposedBy` — enforces maker $\neq$ checker at the engine level. The task is marked unassigned again and the approver is prompted to pick someone else.

- **Completion variables:** when the approver completes the task, the form/API submits `decidedOutcome` (APPROVE or REJECT), `decidedReason` (mandatory if REJECT), `decidedBy` (auto-set to the assignee at completion).

- Exclusive gateway — gw-outcome

- Routes `decidedOutcome == "APPROVE"` to end-approved
- Routes `decidedOutcome == "REJECT"` to end-rejected
- **End event 1** — end-approved (terminates with transition-approved)

- **Execution listener** on end: calls `HomePassStatusTransitionEndListener.onApproved(execution)` — a Java/PHP class registered with Camunda. The listener does the atomic DB write (UPDATE `homepass.status_code`, set `has_been_sellable` if first sellable, INSERT outbox events) and emits the Kafka events. Listener implementation lives in

the homepass module under `homepass-catalog/service/StatusTransitionEndListener.java`.

- **End event 2** — end-rejected (terminates with transition-rejected)
- **Execution listener** on end: calls `HomePassStatusTransitionEndListener.onRejected(execution)`.
HomePass row is NOT updated. The Kafka event `HomePassTransitionRejected` is emitted with the lineage from Camunda history.
- **End event 3** — end-approved-auto (reached when the gateway's NO path fires; same transition-approved end-event name as end-approved)
- Same execution listener as end-approved. From the caller's perspective, this and end-approved are indistinguishable except for the absence of a user-task lineage in Camunda history (which is exactly the audit trail signaling auto-application).

Runtime trace (RFS approval, KE default seed):

```
T+0  StatusTransitionService.startTransition called
      starts process instance with vars

T+0  Camunda enters start event
      gateway gw-needs-approval evaluates ${toStatusCatalogEntry.requires_approval_to_enter}
      RFS catalog row has the flag true (KE seed)
      → user-task path

T+0  Camunda creates task "Approve transition to RFS for HomePass hp_..."
      task-creation listener fires
      notification handler sends email to HOMEPASS_APPROVER group
      Camunda persists task; HTTP 201 returns to caller
      caller (StatusTransitionService) returns 201 to its HTTP caller with state=IN_FLIGHT

T+5m  An approver claims the task
      assignment listener fires
      if claimed by proposer: rejected, task re-opened

T+10m Approver completes with {outcome: APPROVE, reason: "Build certified"}
      gateway gw-outcome routes to end-approved
      end execution listener fires:
        HomePassStatusTransitionEndListener.onApproved
        UPDATE homepass SET status_code='RFS', has_been_sellable=true ...
        INSERT outbox(HomePassStatusChanged, HomePassReachedSellable)
        COMMIT
      Kafka publishes events; downstream onboarding gets the lead-notify trigger
      Camunda process instance terminates
```

Reference BPMN 2: passthrough admin-driven

```
flowchart LR
    Start((Start)) --> EndAuto((End transition-approved))
```

BPMN contains:

- Same process attributes as the strict BPMN (`id="homepass-status-transition"`, same key)
- Same start event with the same 5 input variables
- **No gateway, no user task** — direct sequence flow from start to end
- One **end event** end-approved with the same execution listener as the strict BPMN's end-approved

Runtime trace:

```
T+0  StatusTransitionService.startTransition called
T+0  Camunda enters start event
      sequence flows directly to end-approved
      end execution listener fires synchronously
        UPDATE homepass SET status_code='NSN' ...
        INSERT outbox(HomePassStatusChanged)
        COMMIT
      Kafka publishes the event
      Camunda process instance terminates
      caller returns 201 with state=AUTO_APPLIED, newStatusCode in body
```

End-to-end latency for the passthrough BPMN is roughly the same as a direct DB write — Camunda's in-process execution adds milliseconds, not seconds.

KE configuration

KE ships the **strict** BPMN. The `homepass_status_code` catalog is seeded with the 13 BH codes (NPL, NSR, NSV, NSN, PH1, PH2, PH3, RFC, RFN, RFS, WAI, ACT, RETIRED) and the `requires_approval_to_enter` flag is set as follows

(matching Wananchi's operational practice):

Status code	requires_approval_to_enter	Justification
NPL	false	Initial state, no admin gate
NSR, NSV	false	Documentation states, no commercial impact
NSN, PH1, PH2, PH3	false	Construction-phase states, transitions auto-applied as construction progresses
RFC, RFN	false	Pre-sellable, no commercial gate
RFS	true	Commercially load-bearing — flipping to RFS makes the address sellable; needs technical approval
WAI	true	A subscription is bound; needs operational confirmation that the install order is real
ACT	true	The customer is being marked installed; needs install-team confirmation
RETIRED	true	Terminal, no recovery — needs admin confirmation

The candidate group `HOMEPASS_APPROVER` resolves at KE to the network operations team. The notification template `homepass-transition-pending` resolves to a Wananchi-branded email with the HomePass detail link, the current status, the proposed status, and the proposed reason.

Integration with HOMEPASS-OP-01 (construction workflow)

HOMEPASS-OP-01 is a separate Camunda process (`homepass-construction`) that orchestrates the construction lifecycle — Commercial submits a request → Technical approves → Contractor assigned → MDU formalities → design → build → certify → RFS. At each stage where the HomePass status would change (e.g., approved → NSN, design done → NSN, build certified → RFS), HOMEPASS-OP-01 starts a **child** `homepass-status-transition` process instance via Camunda's **call activity** element. The child workflow runs the deployment's transition policy and reports back. This way, even orchestrated construction flows respect the per-deployment governance.

The call-activity element in HOMEPASS-OP-01:

```
<bpmn:callActivity id="apply-status-transition"
  name="Apply status transition"
  calledElement="homepass-status-transition">
  <bpmn:extensionElements>
    <camunda:in source="homepassId" target="homepassId"/>
    <camunda:in source="currentStatus" target="fromStatusCode"/>
    <camunda:in source="targetStatus" target="toStatusCode"/>
    <camunda:in source="targetCatalogEntry" target="toStatusCatalogEntry"/>
    <camunda:in source="initiator" target="proposedBy"/>
    <camunda:in source="initiatorReason" target="proposedReason"/>
    <camunda:out source="decidedOutcome" target="lastTransitionOutcome"/>
  </bpmn:extensionElements>
</bpmn:callActivity>
```

HOMEPASS-OP-01 then reads `lastTransitionOutcome` to decide its own next step.

Visualisation

```
flowchart LR
  SVC["StatusTransitionService"]
  HPDB[("homepass row")]
  CAM["Camunda process instance"]
  USR["User Task HOMEPASS_APPROVER"]
  LIS["End-event listener"]
  KAFKA[("Kafka events")]
  HIST[("Camunda history")]

  SVC -->|startProcessInstance| CAM
  CAM -.optional.-> USR
  USR --> CAM
  CAM -->|end event| LIS
  CAM -->|persists variables| HIST
  LIS -->|update status_code| HPDB
  LIS -->|emit events| KAFKA
```

Camunda's history (`act_hi_procinst`, `act_hi_taskinst`, `act_hi_varinst`) is the transition ledger. There is no separate transition table in the `homepass` module — see "Status transitions — owned by Camunda, no DB table" in the data model.

Sequence diagrams

Create HomePass (starts in the catalog's `is_initial` status, typically NPL)

```
sequenceDiagram
    participant U as Network Admin
    participant UI as Backoffice UI
    participant HP as homepass module
    participant KIE as KIE Server
    participant DB as homepass DB
    participant K as Kafka

    U->>UI: fill HomePass form
    UI->>HP: POST /api/homepasses
    HP->>HP: derive serviceManagementEndpoints + servicesSupported
    HP->>HP: resolve catalog initial status code
    HP->>KIE: evaluate create rules
    KIE-->>HP: ok
    HP->>DB: INSERT homepass + joins
    HP->>DB: INSERT outbox HomePassCreated
    HP->>K: HomePassCreated event
    HP-->>UI: 201 Created statusCode=NPL
```

Propose status transition into a status with `requires_approval_to_enter = true` (maker side, strict BPMN)

```
sequenceDiagram
    participant U as Network Admin (maker)
    participant HP as homepass module
    participant CAM as Camunda

    U->>HP: POST /transitions toStatusCode=RFS
    HP->>CAM: startProcessInstance with vars
    CAM->>CAM: Gateway requires_approval = true
    CAM-->>HP: process started, user task waiting
    HP-->>U: 201 Accepted state=IN_FLIGHT
```

Note: HomePass.status_code is unchanged at this point. The Camunda process holds a user task for the approver; querying /transitions for this HomePass shows this in-flight process.

Approve a pending transition (checker side; if it's the first transition into a `triggers_lead_notification` status, also fires lead-notify)

```
sequenceDiagram
    participant A as Network Approver
    participant HP as homepass module
    participant CAM as Camunda
    participant DB as homepass DB
    participant K as Kafka

    A->>HP: POST approve transition
    HP->>CAM: completeTask outcome=APPROVE
    CAM->>CAM: Task assignment listener checks approver
    CAM->>CAM: advance to transition-approved
    CAM->>HP: end-event listener invoked
    HP->>DB: UPDATE homepass SET status_code
    HP->>DB: SET has_been_sellable if first sellable
    HP->>DB: INSERT outbox HomePassStatusChanged
    HP->>K: HomePassStatusChanged
    alt has_been_sellable flipped this commit
        HP->>K: HomePassReachedSellable
    end
    HP-->>A: 200 OK newStatusCode=RFS
```

Camunda's history retains the full lineage: who proposed, when, who approved, when, what reason. Querying /api/homepasses/{id}/transitions later projects this from historyService.

Reject a pending transition

```
sequenceDiagram
    participant A as Network Approver
    participant HP as homepass module
    participant CAM as Camunda
    participant K as Kafka

    A->>HP: POST reject transition
    HP->>CAM: completeTask outcome=REJECT
    CAM->>CAM: advance to transition-rejected
    CAM->>HP: end-event listener invoked
    HP->>K: HomePassTransitionRejected
    HP-->>A: 200 OK
```

The rejection lineage (proposer, approver, reason, timestamps) is preserved in Camunda's history — no separate DB row is written by this module.

Auto-applied transition (passthrough BPMN, or strict BPMN on a transition where requires_approval_to_enter = false)

```
sequenceDiagram
    participant U as Network Admin
    participant HP as homepass module
    participant CAM as Camunda
    participant DB as homepass DB
    participant K as Kafka

    U->>HP: POST /transitions toStatusCode=NSN
    HP->>CAM: startProcessInstance with vars
    CAM->>HP: Gateway requires_approval = false
    CAM->>HP: end-event listener invoked
    HP->>DB: UPDATE homepass SET status_code
    HP->>DB: SET has_been_sellable if first sellable
    HP->>DB: INSERT outbox HomePassStatusChanged
    HP->>K: HomePassStatusChanged
    HP-->>U: 200 OK state=AUTO_APPLIED
```

Same shape applies to the passthrough BPMN: every transition reaches transition-approved directly with no user task. The Camunda hop is fast (in-process, no network), so the perceived latency is similar to a direct DB write but the audit lineage in Camunda's history and the event emission go through the same code path as workflows that do gate on approval — uniform.

Bulk import HomePasses

```
sequenceDiagram
    participant U as Bulk Importer
    participant HP as homepass module
    participant KIE as KIE Server
    participant DB as homepass DB
    participant K as Kafka

    U->>HP: POST bulk-import CSV
    HP->>HP: parse CSV, build candidate HomePasses
    loop for each row
        HP->>KIE: evaluate create rules
        KIE-->>HP: ok or errors
    end
    HP->>HP: compile validation report
    alt all rows pass
        HP->>DB: INSERT all rows
        HP->>DB: INSERT outbox HomePassCreated per row
        HP->>K: per-row HomePassCreated events
        HP-->>U: 200 OK rows imported
    else any errors
        HP-->>U: 422 with per-row error report
    end
    end
```

Atomic semantics: either the whole batch commits or none of it does. The user gets a clear per-row error report on failure.

Soft-delete HomePass (cross-module check)

```
sequenceDiagram
    participant U as Network Admin
    participant HP as homepass module
    participant FUL as fulfillment module
    participant KIE as KIE Server
    participant DB as homepass DB
    participant K as Kafka

    U->>HP: DELETE HomePass
    HP->>FUL: GET active services count for HomePass
    FUL-->>HP: count
    HP->>KIE: evaluate R-RLM-CFG-01-H-7 delete rule
    KIE-->>HP: ok or blocked
    alt allowed
        HP->>DB: UPDATE homepass SET deleted_at
        HP->>DB: INSERT outbox
        HP->>K: HomePassRetired
        HP-->>U: 200 OK
    else blocked
        HP-->>U: 409 Conflict with reason
    end
    end
```

Rules (Drools)

PLM-CFG-04 establishes the billing-rules KJAR; RLM-CFG-01 creates a parallel new KJAR.

DRL files

```

modules/homepass-rules/                                ← NEW KJAR
pom.xml                                                  ← produces homepass-rules-{country}_{version}.jar
src/main/java/com/sophix/homepass/facts/
franchise/
    FranchiseFact.java
    FranchiseUniquenessFact.java
    TechRegionActiveCountFact.java
techregion/
    TechRegionFact.java
    TechRegionUniquenessFact.java
    FranchiseRefFact.java
    ActiveHomePassCountFact.java
contractor/
    TechContractorFact.java
    TechContractorSkillFact.java
    TechContractorSkillsExistFact.java
techregion/
    TechRegionFact.java
    TechRegionContractorAssignmentsFact.java
homepass/
    HomePassFact.java
    NetworkPathNode.java
    NodeEndpoint.java
    HomePassAddressUniquenessFact.java
    FranchiseRefFact.java
    TechRegionRefFact.java
    ActiveServicesCountFact.java
    ValidationOutcome.java
src/main/resources/
META-INF/
    kmodule.xml
rules/
    franchise-catalog/
        franchise-validation-create-update.drl
        franchise-validation-retire.drl
        franchise-validation-delete.drl
    tech-contractor-skill-catalog/
        techcontractorskill-validation-create-update.drl
        techcontractorskill-validation-delete.drl
    tech-contractor-catalog/
        techcontractor-validation-create-update.drl
        techcontractor-validation-retire.drl
        techcontractor-validation-delete.drl
    tech-region-catalog/
        techregion-validation-create-update.drl
        techregion-validation-activate.drl ← R-RLM-CFG-01-T-7 (≥1 active contractor)
        techregion-validation-retire.drl
        techregion-validation-delete.drl
    network-node-catalog/
        networknode-validation-create-update.drl
        networknode-validation-retire.drl
        networknode-validation-delete.drl
    homepass-status-code-catalog/
        homepassstatuscode-validation-create-update.drl
        homepassstatuscode-validation-delete.drl
    house-type-catalog/
        housetype-validation-create-update.drl
        housetype-validation-delete.drl
    homepass-catalog/
        homepass-validation-create-update.drl
        homepass-validation-delete.drl
    (NO status-transition DRL – transitions are owned by Camunda)

```

kmodule.xml

```

<kmodule xmlns="http://www.drools.org/xsd/kmodule">
    <kbase name="homepass" packages="rules.franchise-catalog,
                                    rules.tech-contractor-skill-catalog,
                                    rules.tech-contractor-catalog,
                                    rules.tech-region-catalog,
                                    rules.network-node-catalog,
                                    rules.homepass-status-code-catalog,
                                    rules.house-type-catalog,
                                    rules.homepass-catalog">
        <ksession name="homepassSession" type="stateless"/>
    </kbase>
</kmodule>

```

Fact classes (selection — Java, Lombok)

```

package com.sophix.homepass.facts.homepass;

import lombok.Data;

```

```

import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;
import java.util.Map;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class HomePassFact {
    private String id;
    private List<String> franchiseRefs; // many-to-many
    private List<String> techRegionRefs; // many-to-many
    private String accessTechnology; // GPON | HFC | COPPER | FTTH | WIRELESS
    private List<NetworkPathNode> networkPathNodes;
    private String captureMode; // PRE_INSTALLATION | AT_INSTALLATION
    private Map<String, NodeEndpoint> serviceManagementEndpoints; // serviceType → {nodeCode, port}
    private List<String> servicesSupported; // derived: subset of DATA, VOICE, IPTV_MULTICAST
    // Address hierarchy (R-RLM-CFG-01-H-1)
    private String country;
    private String region;
    private String regionL1;
    private String regionL2;
    private String city;
    private String area;
    private String subArea1;
    private String subArea2;
    private String roadName;
    private String buildingNumber;
    private String buildingName;
    private String apartmentNumber;
    private String floor;
    private Integer buildingNumFloors;
    private Integer buildingNumApartments;
    // Classification
    private String propertyType; // RES | COM | MIXED | OTHER | TST
    private boolean ownerOccupied;
    private String houseTypeCode; // FK to house_type catalog
    private int outlets;
    private int activeTerminationPoints;
    // Status (catalog-driven, FK only; flags read via catalog join at service layer)
    private String statusCode; // FK to homepass_status_code
    private boolean hasBeenSellable; // latch for R-RLM-CFG-01-H-6 (HomePass-level history, NOT a catalog flag)
    private String legacyStatusCode; // BH code preserved at migration
    private String notServiceableReason; // e.g., NO_ROE, OWNER_REFUSED
    private LocalDate roeSignedDate; // null if RoE not yet signed
    private String roeDocumentLink; // URL to signed RoE doc, or null
    private String googlePlaceId; // optional GIS Place ID (Google Maps or other adapter); R-RLM-CFG-01-H-1
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class NodeEndpoint {
    private String nodeCode;
    private String port; // nullable
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class NetworkPathNode {
    private String type; // OLT | SPLITTER | FAT | FDT | ONT | HEADEND | DISTRIBUTION_NODE | AMPLIFIER
    private String code;
    private String role; // service_management | passive | termination
    private String port; // nullable; opaque per-deployment identifier
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class HomePassAddressUniquenessFact {
    // Full address tuple per R-RLM-CFG-01-H-1
    private String country;
    private String region;
    private String regionL1;
    private String regionL2;
    private String city;
    private String area;
    private String subArea1;
    private String subArea2;
}

```

```

        private String roadName;
        private String buildingNumber;
        private String buildingName;
        private String apartmentNumber;
        private boolean exists; // is there ANY HomePass with this exact tuple in the deployment?
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public class FranchiseRefFact {
        private String franchiseRef;
        private boolean exists;
        private boolean active;
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public class TechRegionRefFact {
        private String techRegionRef;
        private boolean exists;
        private boolean active;
        private List<String> aggregatedContractorSkills; // union of skills of contractors assigned to this TechRegion – for
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public class TechContractorSkillFact {
        private String skillCode;
        private boolean exists;
        private boolean active; // not soft-deleted
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public class TechContractorFact {
        private String id;
        private String code;
        private String name;
        private List<String> skills; // non-empty per R-RLM-CFG-01-C-3
        private String status;
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public class TechRegionFact {
        private String id;
        private String code;
        private String name;
        private int activeAssignedContractorCount; // for R-RLM-CFG-01-T-7 activation rule
        private String status;
    }

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public class TechContractorSkillsExistFact {
        private String skillCode;
        private boolean exists;
    }

    // StatusTransitionFact has been REMOVED. Transition validity is owned by the
    // Camunda workflow (homepass-status-transition process), not by DRL.

    @Data
    @NoArgsConstructor
    @AllArgsConstructor
    public class NetworkNodeFact {
        private String id;
        private String code;
        private String type; // OLT | SPLITTER | FAT | FDT | ONT | HEADEND | ...
        private String name;
        private String parentNodeCode;
        private String status;
    }

    @Data
    @NoArgsConstructor

```



```

@AllArgsConstructor
public class NetworkNodeExistsFact {
    private String code;
    private boolean exists;
    private boolean active;
    private String type;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class HomePassStatusCodeFact {
    private String code;
    private String description;
    private boolean isInitial;
    private boolean isSellable;
    private boolean isActive;
    private boolean isTerminal;
    private boolean requiresApprovalToEnter;
    private boolean triggersLeadNotification;
    private boolean blocksSoftDelete;
    private String status; // catalog row's own DRAFT/ACTIVE/RETIRED
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class HomePassStatusCodeReferencedFact {
    private String code;
    private boolean referenced; // is there ANY homepass row with status_code = code?
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class HouseTypeFact {
    private String code;
    private String description;
    private String status;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class HouseTypeReferencedFact {
    private String code;
    private boolean referenced;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class ActiveServicesCountFact {
    private String homepassRef;
    private int count;
}

@Data
@NoArgsConstructor
@AllArgsConstructor
public class ValidationOutcome {
    private boolean valid;
    private List<String> errors;
    private List<String> warnings;
}

```

techcontractorskill-validation-create-update.drl

```

package rules.tech-contractor-skill-catalog;

import com.sophix.homepass.facts.contractor.TechContractorSkillFact;
import com.sophix.homepass.facts.homepass.ValidationOutcome;

rule "R-RLM-CFG-01-S-1: TechContractorSkill code must be unique"
when
    $s : TechContractorSkillFact(exists == true)
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-RLM-CFG-01-S-1: TechContractorSkill code '" + $s.getSkillCode() + "' already exists");
    modify($out) { setValid(false); }
end

```

(Immutability of code post-reference — R-RLM-CFG-01-S-2 — and the no-delete-while-referenced rule — R-RLM-CFG-01-S-3 — are enforced at the service layer in TechContractorSkillService by checking the tech_contractor.skills array before allowing the operation. Same pattern as the Service-catalog immutability rules in PLM-CFG-01.)

techcontractor-validation-create-update.drl

```
package rules.tech-contractor-catalog;

import com.sophix.homepass.facts.contractor.TechContractorFact;
import com.sophix.homepass.facts.contractor.TechContractorSkillsExistFact;
import com.sophix.homepass.facts.homepass.ValidationOutcome;

rule "R-RLM-CFG-01-C-3-empty: TechContractor must have at least one skill"
when
    $c : TechContractorFact(skills == null || skills.size() == 0)
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-RLM-CFG-01-C-3: TechContractor must have a non-empty skills list");
    modify($out) { setValid(false); }
end

rule "R-RLM-CFG-01-C-3-each: each skill must exist in the skill catalog"
when
    $c : TechContractorFact($skills : skills != null)
    $s : TechContractorSkillsExistFact(skillCode memberOf $skills, exists == false)
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-RLM-CFG-01-C-3: skill code '" + $s.getSkillCode() + "' does not exist in TechContractorSkillsExistFact");
    modify($out) { setValid(false); }
end
```

(Other TechContractor rules — uniqueness R-RLM-CFG-01-C-1, immutability R-RLM-CFG-01-C-2, retire blocks R-RLM-CFG-01-C-4 — follow the same pattern as Franchise / PLM-CFG-01 service uniqueness; abbreviated here. Soft-delete R-RLM-CFG-01-C-5 is enforced at the service layer by checking tech_region_contractor is empty.)

techregion-validation-activate.drl (R-RLM-CFG-01-T-7)

```
package rules.tech-region-catalog;

import com.sophix.homepass.facts.techregion.TechRegionFact;
import com.sophix.homepass.facts.homepass.ValidationOutcome;

rule "R-RLM-CFG-01-T-7: TechRegion needs ≥1 active assigned contractor to activate"
when
    $t : TechRegionFact(activeAssignedContractorCount == 0)
    $out : ValidationOutcome()
then
    $out.getErrors().add("R-RLM-CFG-01-T-7: cannot activate TechRegion '" + $t.getCode() + "' - no active TechContractor is assigned. Assign at least one contractor first.");
    modify($out) { setValid(false); }
end
```

(Other TechRegion rules — uniqueness, immutability, geojson validation, lifecycle blocks — covered in techregion-validation-create-update.drl and the rest, following the same pattern as Franchise; abbreviated here.)

homepass-validation-create-update.drl

```
package rules.homepass-catalog;

import com.sophix.homepass.facts.homepass.HomePassFact;
import com.sophix.homepass.facts.homepass.NetworkPathNode;
import com.sophix.homepass.facts.homepass.HomePassAddressUniquenessFact;
import com.sophix.homepass.facts.homepass.FranchiseRefFact;
import com.sophix.homepass.facts.homepass.TechRegionRefFact;
import com.sophix.homepass.facts.homepass.ValidationOutcome;

rule "R-RLM-CFG-01-H-1: address uniqueness deployment-wide"
when
    $h : HomePassFact()
    $u : HomePassAddressUniquenessFact(
        country      == $h.country,
        region       == $h.region,
        regionL1     == $h.regionL1,
        regionL2     == $h.regionL2,
        city         == $h.city,
        area         == $h.area,
        subArea1     == $h.subArea1,
        subArea2     == $h.subArea2,
        roadName     == $h.roadName,
```

```

        buildingNumber == $.buildingNumber,
        buildingName == $.buildingName,
        apartmentNumber == $.apartmentNumber,
        exists == true)
    $out : ValidationOutcome()
  then
    $out.getErrors().add("R-RLM-CFG-01-H-1: HomePass at " + $.getRoadName()
      + (($.getBuildingNumber() != null) ? " " + $.getBuildingNumber() : "")
      + (($.getApartmentNumber() != null) ? " apt " + $.getApartmentNumber() : "")
      + " already exists in this deployment");
    modify($out) { setValid(false); }
  end

end

rule "R-RLM-CFG-01-H-2a-empty: franchiseRefs must be non-empty"
  when
    $h : HomePassFact(franchiseRefs == null || franchiseRefs.size() == 0)
    $out : ValidationOutcome()
  then
    $out.getErrors().add("R-RLM-CFG-01-H-2a: HomePass must have at least one franchiseRef");
    modify($out) { setValid(false); }
  end

end

rule "R-RLM-CFG-01-H-2a-each: each franchiseRef must exist and be ACTIVE"
  when
    $h : HomePassFact($refs : franchiseRefs != null)
    $f : FranchiseRefFact(franchiseRef memberOf $refs, (exists == false || active == false))
    $out : ValidationOutcome()
  then
    String msg = "R-RLM-CFG-01-H-2a: franchiseRef '" + $f.getFranchiseRef() + "' ";
    msg += (!$f.isExists()) ? "does not exist" : "is not ACTIVE";
    $out.getErrors().add(msg);
    modify($out) { setValid(false); }
  end

end

rule "R-RLM-CFG-01-H-2b-empty: techRegionRefs must be non-empty"
  when
    $h : HomePassFact(techRegionRefs == null || techRegionRefs.size() == 0)
    $out : ValidationOutcome()
  then
    $out.getErrors().add("R-RLM-CFG-01-H-2b: HomePass must have at least one techRegionRef");
    modify($out) { setValid(false); }
  end

end

rule "R-RLM-CFG-01-H-2b-each: each techRegionRef must exist and be ACTIVE"
  when
    $h : HomePassFact($refs : techRegionRefs != null)
    $t : TechRegionRefFact(techRegionRef memberOf $refs, (exists == false || active == false))
    $out : ValidationOutcome()
  then
    String msg = "R-RLM-CFG-01-H-2b: techRegionRef '" + $t.getTechRegionRef() + "' ";
    msg += (!$t.isExists()) ? "does not exist" : "is not ACTIVE";
    $out.getErrors().add(msg);
    modify($out) { setValid(false); }
  end

end

rule "R-RLM-CFG-01-H-3: accessTechnology must be in the allowed set"
  when
    $h : HomePassFact(
      accessTechnology != "GPON" &&
      accessTechnology != "HFC" &&
      accessTechnology != "COPPER" &&
      accessTechnology != "FTTH" &&
      accessTechnology != "WIRELESS")
    $out : ValidationOutcome()
  then
    $out.getErrors().add("R-RLM-CFG-01-H-3: accessTechnology must be GPON | HFC | COPPER | FTTH | WIRELESS (got '" + $h.getAccessTechnology() + "')");
    modify($out) { setValid(false); }
  end

end

rule "R-RLM-CFG-01-H-9: activeTerminationPoints must be non-negative"
  when
    $h : HomePassFact(activeTerminationPoints < 0)
    $out : ValidationOutcome()
  then
    $out.getErrors().add("R-RLM-CFG-01-H-9: activeTerminationPoints must be ≥ 0, got " + $.getActiveTerminationPoints());
    modify($out) { setValid(false); }
  end

end

rule "R-RLM-CFG-01-H-10a: networkPath must have at least one node"
  when
    $h : HomePassFact(networkPathNodes == null || networkPathNodes.size() == 0)
    $out : ValidationOutcome()
  then
    $out.getErrors().add("R-RLM-CFG-01-H-10a: networkPath must have at least one node");
    modify($out) { setValid(false); }
  end

end

```

```

        then
            $out.getErrors().add("R-RLM-CFG-01-H-10: networkPath must contain at least one node");
            modify($out) { setValid(false); }
        end

rule "R-RLM-CFG-01-H-10b: captureMode must be PRE_INSTALLATION or AT_INSTALLATION"
    when
        $h : HomePassFact(captureMode != "PRE_INSTALLATION" && captureMode != "AT_INSTALLATION")
        $out : ValidationOutcome()
    then
        $out.getErrors().add("R-RLM-CFG-01-H-10: captureMode must be PRE_INSTALLATION or AT_INSTALLATION (got '" + $h.getCaptureMode() + "')");
        modify($out) { setValid(false); }
    end

rule "R-RLM-CFG-01-H-10c: each node type must be in the allowed set"
    when
        $h : HomePassFact($nodes : networkPathNodes != null)
        $n : NetworkPathNode(
            this memberOf $nodes,
            type != "OLT" && type != "SPLITTER" && type != "FAT" && type != "FDT" &&
            type != "ONT" && type != "HEADEND" && type != "DISTRIBUTION_NODE" &&
            type != "AMPLIFIER" && type != "LINE_EXTENDER" && type != "VOIPSWITCH" &&
            type != "NMS" && type != "OTHER")
        $out : ValidationOutcome()
    then
        $out.getErrors().add("R-RLM-CFG-01-H-10: invalid network-path node type '" + $n.getType() + "'");
        modify($out) { setValid(false); }
    end

rule "R-RLM-CFG-01-H-10d: each node role must be service_management, passive, or termination"
    when
        $h : HomePassFact($nodes : networkPathNodes != null)
        $n : NetworkPathNode(
            this memberOf $nodes,
            role != "service_management" && role != "passive" && role != "termination")
        $out : ValidationOutcome()
    then
        $out.getErrors().add("R-RLM-CFG-01-H-10: invalid network-path node role '" + $n.getRole() + "' (node code '" + $n.getCode() + "')");
        modify($out) { setValid(false); }
    end

rule "R-RLM-CFG-01-H-13: when present, port must be non-empty and at most 64 chars"
    when
        $h : HomePassFact($nodes : networkPathNodes != null)
        $n : NetworkPathNode(
            this memberOf $nodes,
            port != null,
            (port == "" || port.length() > 64))
        $out : ValidationOutcome()
    then
        $out.getErrors().add("R-RLM-CFG-01-H-13: invalid port '" + $n.getPort() + "' on node '" + $n.getCode() + "' - must be non-empty and at most 64 chars");
        modify($out) { setValid(false); }
    end

rule "R-RLM-CFG-01-H-8: franchise/techregion refs are append-only once status is_active or blocks_soft_delete"
    when
        $h : HomePassFact()
        // Status flags joined in by the service layer before rule fire:
        $sc : HomePassStatusCodeFact(code == $h.statusCode, isActive == true || blocksSoftDelete == true)
        $out : ValidationOutcome()
        eval(false)
    then
        // Append-only constraint enforced at service layer: compare incoming refs lists vs stored
        // and reject if any existing ref has been removed. New refs may be added.
    end
end

```

Note: R-RLM-CFG-01-H-11 (serviceManagementEndpoints derivation) is computed in the service layer before the rules engine is called, not enforced by DRL — it's a derivation rule, not a validation rule. The Drools-side responsibility is to validate the networkPath structure (R-RLM-CFG-01-H-10, R-RLM-CFG-01-H-13); the derivation lives in HomePassService Java code where the JSONB-to-map mapping logic is more natural to express.

There is **no** homepass-validation-status-transition.drl file. Transition validity is owned by the Camunda workflow (homepass-status-transition process key — see "Status workflow" section). R-RLM-CFG-01-H-4 (status code references the catalog) is enforced as a service-layer foreign-key check on homepass_status_code; R-RLM-CFG-01-H-5 (maker-checker) is enforced inside the workflow's user-task assignment listener; R-RLM-CFG-01-H-6 (first-sellable event emission) is enforced as a service-layer listener on the workflow's transition-approved end event, gated by the has_been_sellable boolean latch on the HomePass row.

(Build/deploy of the homepass-rules KJAR mirrors PLM-CFG-01's reference section — Maven build per country, Maven repo publish, KIE Server pulls latest at startup, hot-reload via KIE Scanner.)

Events (Kafka)

Five topic families. Key formats: homepass:franchise:{id}, homepass:techcontractorskill:{id}, homepass:techcontractor:{id}, homepass:techregion:{id}, homepass:homepass:{id}.

sophix.homepass.homepass.created

```
{
  "eventId": "homepass:homepass:hp_01HX:evt_01JZ...",
  "eventType": "HomePassCreated",
  "aggregateId": "hp_01HX...",
  "timestamp": "2026-05-12T10:00:00Z",
  "actor": "network.admin@wananchi.co.ke",
  "correlationId": "corr_01HX...",
  "payload": {
    "homepassId": "hp_01HX...",
    "franchiseRefs": ["fr_002_Nairobi"],
    "techRegionRefs": ["tr_005_Westlands"],
    "accessTechnology": "GPON",
    "networkPath": {
      "captureMode": "PRE_INSTALLATION",
      "nodes": [
        { "type": "OLT", "code": "OLT-NRB-WTL-01", "role": "service_management", "port": "1/2/3" },
        { "type": "SPLITTER", "code": "SPL-NRB-WTL-FAT-12", "role": "passive", "port": "P05" },
        { "type": "FAT", "code": "FAT-NRB-WTL-12-A4", "role": "termination", "port": "A4-7" }
      ]
    },
    "serviceManagementEndpoints": {
      "data": { "nodeCode": "OLT-NRB-WTL-01", "port": "1/2/3" }
    },
    "servicesSupported": ["DATA"],
    "address": {
      "country": "KE",
      "region": null,
      "regionL1": "Nairobi County",
      "regionL2": "Westlands Sub-County",
      "city": "Nairobi",
      "area": "Kilimani",
      "subArea1": null,
      "subArea2": null,
      "roadName": "Wood Avenue",
      "buildingNumber": "12",
      "buildingName": "Finewood Apartments",
      "apartmentNumber": "M18",
      "floor": "M18",
      "buildingNumFloors": 13,
      "buildingNumApartments": 48
    },
    "propertyType": "RES",
    "houseTypeCode": "M2M",
    "statusCode": "NPL",
    "statusFlags": {
      // snapshot of catalog flags at event time;
      // consumers can use this directly OR re-resolve
      // by reading the catalog. Flags are NOT stored
      // on the homepass row.
      "isSellable": false,
      "isActive": false,
      "isTerminal": false
    }
  }
}
```

sophix.homepass.homepass.statuschanged

Emitted whenever a HomePass status flips (after the workflow's transition-approved end event fires).

```
{
  "eventId": "homepass:homepass:hp_01HX:evt_01JZ7...",
  "eventType": "HomePassStatusChanged",
  "aggregateId": "hp_01HX...",
  "timestamp": "2026-05-12T11:00:00Z",
  "actor": "approver@wananchi.co.ke",
  "correlationId": "corr_01HX...",
  "payload": {
    "homepassId": "hp_01HX...",
    "previousStatusCode": "NSN",
    "newStatusCode": "RFS",
  }
}
```

```

    "newStatusFlags": {
      "isSellable": true,
      "isActive": false,
      "isTerminal": false,
      "triggersLeadNotification": true,
      "blocksSoftDelete": false
    },
    "processInstanceId": "cam_proc_01HX...",
    "decidedBy": "approver@wananchi.co.ke",
    "proposedBy": "maker@wananchi.co.ke",
    "autoApplied": false
  }
}

```

sophix.homepass.homepass.transitionrejected

Emitted when the workflow's transition-rejected end event fires. HomePass status unchanged.

```

{
  "eventType": "HomePassTransitionRejected",
  "payload": {
    "homepassId": "hp_01HX...",
    "fromStatusCode": "NSN",
    "toStatusCode": "RFS",
    "processInstanceId": "cam_proc_01HX...",
    "proposedBy": "maker@wananchi.co.ke",
    "decidedBy": "approver@wananchi.co.ke",
    "decidedReason": "build certification not yet provided"
  }
}

```

sophix.homepass.homepass.reachedsellable

Emitted once per HomePass when it first reaches RFS (workflow-path independent). Drives the lead-notification flow.

```

{
  "eventId": "homepass:homepass:hp_01HX:evt_01JZ8...",
  "eventType": "HomePassReachedSellable",
  "aggregateId": "hp_01HX...",
  "timestamp": "2026-05-12T11:00:00Z",
  "actor": "approver@wananchi.co.ke",
  "correlationId": "corr_01HX...",
  "payload": {
    "homepassId": "hp_01HX...",
    "statusCode": "RFS",
    "address": {
      "country": "KE",
      "regionL1": "Nairobi County",
      "city": "Nairobi",
      "area": "Kilimani",
      "roadName": "Wood Avenue",
      "buildingNumber": "12",
      "buildingName": "Finewood Apartments",
      "apartmentNumber": "M18"
    },
    "franchiseRefs": ["fr_002_Nairobi"],
    "techRegionRefs": ["tr_005_Westlands"],
    "accessTechnology": "GPON",
    "servicesSupported": ["DATA"]
  }
}

```

Other event topics (same envelope shape with respective payloads)

- sophix.homepass.franchise.created / updated / activated / retired
- sophix.homepass.techcontractorskill.created / updated / retired
- sophix.homepass.techcontractor.created / updated / activated / retired
- sophix.homepass.techregion.created / updated / activated / retired
- sophix.homepass.techregion.contractor_assigned / contractor_unassigned (payload: {techRegionRef, techContractorRef, actor, timestamp})
- sophix.homepass.networknode.created / updated / activated / retired
- sophix.homepass.homepassstatuscode.created / updated / activated / retired
- sophix.homepass.housetype.created / updated / activated / retired

- `sophix.homepass.homepass.updated` (any non-address, non-status field change, including `network_path` updates from the `append/ports` endpoints)
- `sophix.homepass.homepass.addresscorrected` (payload: `{homepassId, before: {country, region_l1, ..., apartmentNumber}, after: {...}, actor, reasonNote?}` — emitted specifically when address fields change so downstream subscribers — search, sales UI, mapping — can refresh address-dependent caches deterministically)
- `sophix.homepass.homepass.geoenriched` (payload: `{homepassId, before: {latitude, longitude, country, region_l1, ..., googlePlaceId}, after: {...}, source: "geocode|reverse_geocode|place_details|contains", actor}` — emitted when the optional GIS enrichment flow fills previously-null fields. Separate from `addresscorrected` because the semantics differ: enrichment populates missing data from a GIS provider, whereas correction overrides existing data based on a manual fix. Downstream subscribers can treat them differently if needed.)
- `sophix.homepass.homepass.retired / deleted`

Exceptions

Exception	When	HTTP
<code>HomePassValidationException</code>	Rules failed (create/update/transition)	422
<code>HomePassNotFoundException</code>	GET / action on unknown id	404
<code>HomePassConflictException</code>	State-machine violation, maker=checker, etc.	409
<code>BulkImportException</code>	One or more rows in a bulk import failed validation	422 (whole batch rejected)
<code>CrossModuleRefException</code>	Franchise / TechRegion / external ref invalid	422
<code>AuthorizationException</code>	Caller missing required role	403

APIs

All under `/api/franchises`, `/api/tech-regions`, `/api/homepasses`. Bearer JWT.

Status codes (standard taxonomy per HLD)

Code	Meaning
200 / 201 / 202 / 400 / 401 / 403 / 404 / 409 / 422 / 500	Same semantics as PLM-CFG-04

Franchise endpoints (CRUD + lifecycle)

- GET `/api/franchises` — list; query params: `status`, `page/size`
- GET `/api/franchises/{id}` — read one
- POST `/api/franchises` — create (DRAFT)
- PATCH `/api/franchises/{id}` — update (limited fields editable after first ACTIVE)
- POST `/api/franchises/{id}/activate` — DRAFT → ACTIVE
- POST `/api/franchises/{id}/retire` — ACTIVE → RETIRED (R-RLM-CFG-01-F-3 enforced)
- DELETE `/api/franchises/{id}` — soft-delete (R-RLM-CFG-01-F-4 enforced)

TechContractorSkill endpoints (CRUD)

- GET `/api/tech-contractor-skills` — list; query params: `status`, `page/size`
- GET `/api/tech-contractor-skills/{id}` — read one
- POST `/api/tech-contractor-skills` — create. Body: `{code, name, description}`. Code uppercase, unique.
- PATCH `/api/tech-contractor-skills/{id}` — update name/description (code immutable once referenced — R-RLM-CFG-01-S-2)
- DELETE `/api/tech-contractor-skills/{id}` — soft-delete (R-RLM-CFG-01-S-3 enforced — no `TechContractor` references it)

TechContractor endpoints (CRUD + lifecycle)

- GET `/api/tech-contractors` — list; query params: `status`, `skill` (return contractors whose `skills[]` contains this code), `page/size`

- GET /api/tech-contractors/{id} — read one
- POST /api/tech-contractors — create (DRAFT). Body: {code, name, description, skills[]}. skills is non-empty.
- PATCH /api/tech-contractors/{id} — update; code immutable post-DRAFT
- POST /api/tech-contractors/{id}/activate — DRAFT → ACTIVE
- POST /api/tech-contractors/{id}/retire — R-RLM-CFG-01-C-4 enforced (no active HomePass behind this contractor's TechRegions)
- DELETE /api/tech-contractors/{id} — soft-delete (R-RLM-CFG-01-C-5 enforced — tech_region_contractor empty)

TechRegion endpoints

- GET /api/tech-regions — list; query params: status, skill (return TechRegions whose assigned contractors collectively cover this skill — server-side join + filter), page/size. Note: there is NO skills column on TechRegion; the filter walks the contractor assignments.
- GET /api/tech-regions/{id} — response includes assignedContractors[] summary (id, code, skills)
- POST /api/tech-regions — create (DRAFT). Body: {code, name, description, boundaryGeoJson?}. No skills field — skills live on the per-contractor assignment row (tech_region_contractor.skills[]).
- PATCH /api/tech-regions/{id} — update; code immutable post-DRAFT
- POST /api/tech-regions/{id}/activate — DRAFT → ACTIVE (R-RLM-CFG-01-T-7 enforced: ≥1 active contractor must be assigned)
- POST /api/tech-regions/{id}/retire — R-RLM-CFG-01-T-3 enforced
- DELETE /api/tech-regions/{id} — soft-delete (R-RLM-CFG-01-T-4 enforced)

Contractor assignment to TechRegion

- POST /api/tech-regions/{id}/contractors — assign one or more contractors. Body: {contractorIds: ["tc_01HX...", ...]}. Inserts rows into tech_region_contractor. Each referenced contractor must exist and be ACTIVE.
- DELETE /api/tech-regions/{id}/contractors/{contractorId} — unassign. Service-layer check: the TechRegion must retain at least one active contractor after the unassignment if it is itself ACTIVE; otherwise 409 (a-la R-RLM-CFG-01-T-7).
- GET /api/tech-regions/{id}/contractors — list assigned contractors with their skills

HomePass endpoints

- GET /api/homepasses — list / search; query params: franchiseRef, techRegionRef, status, country, regionL1, regionL2, regionL3, area, roadName (partial match), buildingName (partial), apartmentNumber, accessTechnology, servicesSupported, networkNodeCode (filter HomePasses whose network_path contains this node code anywhere), networkNodePort (filter HomePasses whose path contains a node with this specific port — useful for "which HomePass is on OLT port 1/2/3?"), page/size. The set of address filters mirrors the BH search dialog (Road / Building# / Building Name / Apt / Area / City) generalised to the full hierarchy.
- GET /api/homepasses/{id} — read one. Response carries franchiseRefs[], techRegionRefs[], servicesSupported[], the full address hierarchy, and full networkPath.nodes[] with ports.
- GET /api/homepasses/eligible?franchiseRef=...&country=...®ionL1=...&roadName=...&buildingNumber=...&buildingName=...&apartmentNumber=... — **hot acquisition endpoint**: returns HomePass(es) matching the given address tuple whose status is RFS or WAI (sales-eligible) AND whose franchiseRefs[] contains the requested Franchise. **For HomePasses that are currently ACT** (i.e., a customer is already installed), the response includes the bound subscription summary {subscriptionId, customerName} so the sales-team UX can show "this address is already taken by X" rather than leaving the agent guessing. The subscription detail is fetched cross-module from fulfillment (or subscription module — TBD); a cache layer absorbs the second hop so the eligible-lookup latency stays low. Cached aggressively.
- GET /api/homepasses/{id}/eligible-contractors?skill=INSTALLATION&status=ACTIVE — **lookup primitive for WO routing**. Returns the TechContractors that (a) serve any TechRegion linked to this HomePass, AND (b) have the requested skill in their skills[], AND (c) match the requested status (default ACTIVE). The endpoint is the integration point for the future WO DD; RLM-CFG-01 does NOT prescribe how WOs use the result (manual pick vs auto-assign vs slot-capacity check — all owned by the WO DD).

- POST /api/homepasses — create (status NPL). Body must include franchiseRefs[], techRegionRefs[], the full address hierarchy (country, regionL1, roadName required; others optional), and networkPath (with captureMode and nodes[] including ports where known). serviceManagementEndpoints and servicesSupported are derived server-side, not accepted in the request.
- PATCH /api/homepasses/{id} — update non-status fields. franchiseRefs[] and techRegionRefs[] are append-only once HomePass is in a status with is_active or blocks_soft_delete (R-RLM-CFG-01-H-8 — typically WAI/ACT). Address fields ARE mutable post-create (R-RLM-CFG-01-H-14) — emits HomePassAddressCorrected. serviceManagementEndpoints and servicesSupported re-derived on any networkPath change.
- PATCH /api/homepasses/{id}/network-path/append — **for AT_INSTALLATION captureMode regions**: called by fulfillment (or the WO module at install) to append last-mile nodes (specific ONT, port assignment, etc.). Body: {nodes: [{type, code, role, port}, ...]}. Service-layer enforces that this endpoint is only callable when the HomePass's captureMode = AT_INSTALLATION (else 409). Each appended node's code must resolve to an ACTIVE network_node (R-RLM-CFG-01-H-15). serviceManagementEndpoints and servicesSupported are re-derived. A HomePassUpdated event is emitted with the new full path.
- PATCH /api/homepasses/{id}/network-path/ports — **port-only update** for in-place port corrections (e.g., a port number was wrong in the original creation). Body: {nodePortUpdates: [{nodeCode, port}, ...]}. Each entry updates the matching node's port. serviceManagementEndpoints re-derived if any service-management node's port changes. Audited; emits HomePassUpdated.
- POST /api/homepasses/{id}/enrich-from-geo — **optional**, requires GIS configured. Reverse-geocodes from latitude/longitude (if present) to fill any null admin-hierarchy fields, and/or forward-geocodes from the address tuple to fill latitude/longitude if null. Sets google_place_id (or other provider Place ID) if it was previously null and the geocoder returned one (R-RLM-CFG-01-H-18 makes it immutable thereafter). Optionally invokes the geo-client adapter contains method to suggest franchiseRefs[] and techRegionRefs[] based on polygon containment — admin reviews and applies. Returns the post-enrichment HomePass plus a diff showing what changed. Audited; emits HomePassGeoEnriched and (if address fields changed) HomePassAddressCorrected.
- GET /api/homepasses/map?bbox=W,S,E,N&zoom=N&statusFilter=...&franchiseRef=...&accessTechnology=... — **optional**, drives the Backoffice map view. Returns a GeoJSON FeatureCollection of HomePass points within the bounding box, filtered. Each Feature: {type: "Feature", geometry: {type: "Point", coordinates: [lng, lat]}, properties: {id, statusCode, accessTechnology, buildingName, houseTypeCode}}. HomePasses with null coordinates are excluded. Caller is expected to tile by bbox at high zoom levels.
- POST /api/homepasses/{id}/transitions — propose status transition. Body: {toStatusCode, reason?}. The service layer starts a Camunda process under key homepass-status-transition with variables {homepassId, fromStatusCode, toStatusCode, toStatusCatalogEntry, proposedBy, proposedReason}. Returns 201 with {processInstanceId, state, currentStatusCode}. state is one of IN_FLIGHT (workflow waiting on user task), AUTO_APPLIED (workflow completed synchronously, status already flipped), or REJECTED (workflow rejected synchronously). The decision gateway in the strict BPMN reads toStatusCatalogEntry.requires_approval_to_enter to decide.
- POST /api/homepasses/{id}/transitions/{processInstanceId}/approve — completes the pending approval user task on that Camunda process instance with APPROVE. Camunda advances to the transition-approved end event; the end-event listener updates HomePass status and emits events. Returns 200 with the post-flip HomePass status code.
- POST /api/homepasses/{id}/transitions/{processInstanceId}/reject — completes the pending approval user task with REJECT (reason required). Camunda advances to the transition-rejected end event. Returns 200; HomePass status unchanged.
- GET /api/homepasses/{id}/transitions — transition history for one HomePass. Implementation queries Camunda's history API: historyService.createHistoricProcessInstanceQuery().processDefinitionKey("homepass-status-transition").variableValueEquals("homepassId", id).orderByProcessInstanceEndTime().desc(). Returns a list of historic process instances projected to a stable response shape: [{processInstanceId, fromStatusCode, toStatusCode, state, proposedBy, proposedAt, decidedBy, decidedAt, decidedOutcome, decidedReason}]. State is derived from process status + end event reached.
- GET /api/homepasses/transitions/pending — approver queue. Queries Camunda's task list filtered by the caller's candidate group (typically HOMEPASS_APPROVER) and the process definition key. Returns [{processInstanceId, homepassId, toStatusCode, proposedBy, proposedAt, taskId}]. The caller uses processInstanceId in the approve/reject URLs above.

- POST /api/homepasses/bulk-import — bulk import (CSV multipart); body of response is validation report
- POST /api/homepasses/{id}/clone — **clone a HomePass** to bulk-create siblings (typical use: 48 apartments in one building share most fields, vary only apartment number). Body: {count: 47, varyField: "apartmentNumber", varyValues: ["02", "03", ..., "48"], overrides?: {networkPath: {...}, ...}}. Each clone deep-copies the source, overrides the varying field per entry, applies optional overrides, gets a fresh ULID, forces statusCode to the deployment's initial code, resets has_been_sellable = false, clears audit fields. Returns the array of new HomePass IDs and any per-row validation errors. Mirrors BH's "Clone From" address-dialog feature, extended to clone the full HomePass.
- DELETE /api/homepasses/{id} — soft-delete (R-RLM-CFG-01-H-7 enforced; cross-module check vs fulfillment; service layer joins to homepass_status_code to check blocks_soft_delete = false on the current status)

NetworkNode endpoints (CRUD + lifecycle)

- GET /api/network-nodes — list; query params: status, type (filter by OLT / FAT / etc.), parentNodeCode (children of a given node), page/size
- GET /api/network-nodes/{id} — read one (with parent chain if requested via ?withParents=true)
- POST /api/network-nodes — create (DRAFT). Body: {code, type, name, parentNodeCode?, description?, metadata?}
- PATCH /api/network-nodes/{id} — update; code and type immutable post-DRAFT
- POST /api/network-nodes/{id}/activate — DRAFT → ACTIVE
- POST /api/network-nodes/{id}/retire — R-RLM-CFG-01-N-5 enforced
- DELETE /api/network-nodes/{id} — soft-delete (R-RLM-CFG-01-N-6 enforced)
- POST /api/network-nodes/bulk-import — bulk CSV import for the initial catalog seed

HomePassStatusCode endpoints (CRUD + lifecycle)

Per-deployment configurable catalog of HomePass status codes. Seeded at deployment provisioning time with the default set or a custom set (Wananchi KE seeds with BH's richer code set).

- GET /api/homepass-status-codes — list; query params: status (the catalog row's own DRAFT/ACTIVE/RETIRED), filter flags (isSellable=true, isInitial=true, etc.), page/size
- GET /api/homepass-status-codes/{id} — read one
- POST /api/homepass-status-codes — create (DRAFT). Body: {code, description, displayOrder, isInitial?, isSellable?, isActive?, isTerminal?, requiresApprovalToEnter?, triggersLeadNotification?, blocksSoftDelete?}. Booleans default to false. R-RLM-CFG-01-HS-3 (single is_initial), R-RLM-CFG-01-HS-5 (terminal mutex) enforced.
- PATCH /api/homepass-status-codes/{id} — update; code immutable once referenced by any HomePass (R-RLM-CFG-01-HS-2). The flags ARE mutable — flag changes take effect immediately on the next read, since the HomePass row only carries the FK status_code and the flags are resolved via JOIN at query time. An audit event (HomePassStatusCodeUpdated) is emitted so downstream caches that mirror the catalog can refresh.
- POST /api/homepass-status-codes/{id}/activate — DRAFT → ACTIVE
- POST /api/homepass-status-codes/{id}/retire — R-RLM-CFG-01-HS-4 enforced (no HomePass references)
- DELETE /api/homepass-status-codes/{id} — soft-delete (R-RLM-CFG-01-HS-4 enforced)

HouseType endpoints (CRUD + lifecycle)

- GET /api/house-types — list; query params: status, page/size
- GET /api/house-types/{id} — read one
- POST /api/house-types — create (DRAFT). Body: {code, description, displayOrder?}
- PATCH /api/house-types/{id} — update; code immutable once referenced (R-RLM-CFG-01-HT-2)
- POST /api/house-types/{id}/activate — DRAFT → ACTIVE
- POST /api/house-types/{id}/retire
- DELETE /api/house-types/{id} — soft-delete (R-RLM-CFG-01-HT-3 enforced)

Example: propose status transition

Request:

```
POST /api/homepasses/hp_01HX/transitions
{
  "toStatusCode": "RFS",
  "reason": "Build complete, design map signed off"
}
```

Response 201 (workflow started, awaiting approval):

```
{
  "processInstanceId": "cam_proc_01HX...",
  "homepassId": "hp_01HX...",
  "fromStatusCode": "NSN",
  "toStatusCode": "RFS",
  "state": "IN_FLIGHT",
  "requiresApproval": true,
  "proposedBy": "maker@wananchi.co.ke",
  "proposedAt": "2026-05-12T11:00:00Z"
}
```

(For workflows that auto-apply with no user task — e.g., the passthrough BPMN, or the strict BPMN on a transition into a status whose `requires_approval_to_enter` flag is false — the workflow completes synchronously and the response carries `state: "AUTO_APPLIED"`, `requiresApproval: false`, and `newStatusCode` reflecting the already-flipped status. The HTTP semantics are the same; only the response state differs. `requiresApproval` is derived at response time from the `toStatus` catalog entry, purely informational.)

Example: approve

Request:

```
POST /api/homepasses/hp_01HX/transitions/cam_proc_01HX/approve
```

Response 200:

```
{
  "processInstanceId": "cam_proc_01HX...",
  "homepassId": "hp_01HX...",
  "state": "APPROVED",
  "decidedBy": "approver@wananchi.co.ke",
  "decidedAt": "2026-05-12T11:05:00Z",
  "newStatusCode": "RFS"
}
```

Response 422 if `approver == proposer` (rejected by the BPMN's user-task assignment listener — applies only when the deployment's BPMN includes this guard, e.g., the strict reference BPMN):

```
{ "errors": ["WORKFLOW_TASK_ASSIGNMENT: approver must differ from proposer per the configured BPMN"] }
```

Example: bulk import

```
POST /api/homepasses/bulk-import
Content-Type: multipart/form-data; boundary=...
File field: homepasses.csv
```

CSV columns (initial schema, extensible): `franchiseCodes`, `techRegionCodes`, `accessTechnology`, `captureMode`, `networkPathJson`, `country`, `region`, `regionL1`, `regionL2`, `city`, `area`, `subArea1`, `subArea2`, `roadName`, `buildingNumber`, `buildingName`, `apartmentNumber`, `floor`, `buildingNumFloors`, `buildingNumApartments`, `propertyType`, `ownerOccupied`, `houseTypeCode`, `outlets`, `activeTerminationPoints`, `latitude`, `longitude`, `altitude`, `mapCode`, `statusCode`, `legacyStatusCode`, `notServiceableReason`, `permDate`, `surveyDate`, `roeSignedDate`, `roeDocumentLink`, `googlePlaceId`, `directions`, `comments`, `notes`

`franchiseCodes` and `techRegionCodes` are **semicolon-separated lists** of codes (semicolon avoids conflict with CSV comma). Example cell value: `002;B2B-NRB`. The importer resolves each code to the corresponding row id, validates all refs exist and are ACTIVE, then writes the join rows. For BH-imported rows (which carry a single Franchise + single TechRegion per HomePass), the cell holds one code each; admins extend the lists manually after import if needed.

The `networkPathJson` cell carries the full network-path JSON including each node's port. Every node code must resolve against the pre-seeded `network_node` catalog — bulk-importing HomePasses BEFORE seeding the `NetworkNode` catalog will fail rows with "unknown node code" errors in the validation report.

The `networkPathJson` cell holds the full `network_path` JSON for that row (or an escaped JSON string). For BH-imported rows, the importer constructs this from BH Head End, Hub Number, Node, Drop Tag fields per a documented mapping (see migration section). `serviceManagementEndpoints` is derived server-side per row.

Response 200 (success):

```
{
  "rowsImported": 1247,
  "rowsRejected": 0,
  "homepassIds": ["hp_01HX...", "hp_01HY...", "..."]
}
```

Response 422 (one or more rejected, nothing committed):

```
{
  "rowsImported": 0,
  "rowsRejected": 3,
  "rowErrors": [
    { "row": 42, "errors": ["R-RLM-CFG-01-H-2: franchiseRef 'fr_999' does not exist"] },
    { "row": 128, "errors": ["R-RLM-CFG-01-H-1: HomePass at Wood Ave 12 apt M18 already exists in franchise fr_002"] },
    { "row": 301, "errors": ["R-RLM-CFG-01-H-3: accessTechnology must be in the allowed set (got 'XPON')"] }
  ]
}
```

B — Current TZ

TZ Sophix has **no equivalent** of HomePass / Franchise / TechRegion. The catalog inventory marks this entire row as ■ New.

Wananchi KE currently uses **Broadhub's Home Passed module** (see workshops 2026-04-29 HPRFS deep-dive).

Concretely, what exists today in KE Broadhub:

- **Home Passed screen** with sections Provisioning (Fiber node / Amplifier node / Line extender / Head End / Hub Number / Outlets / Signal / Node / Drop Tag), Region (Tech Region / Sales Agent / Tech / Floor), Home (Building / Property Type / Owner occupied), Misc (Status NPL / NSN / RFS / WAI / ACT / Franchise / House Type / Perm Date / Survey Date)
- **Homes Modification Utility** for bulk edits
- **Home Change History** for audit
- **Homes Passed Data Import dialog** (Schema NS2) — broken at workshop time (returns errors during demo)
- **Construction approval portal** (construction-approval.wananchi.com) — multi-stage approval workflow before BH HomePass creation
- **MDU platform** — owner formalities (Right-of-Entry, agreement, design map review, address confirmation)
- **AutoCAD file server** with design files
- **malogs.wananchi.com GPON Auto Find** — discovered ONTs by F/S/P + Ont SN

What's already well-defined in KE workflow (we keep this):

- The status set: NPL / NSN / RFS / WAI / ACT (some others noted in workshops — "Sekou action item to confirm eligible-for-sales list")
- Two-person access control on HomePass edits (an operational discipline established by the technical team to limit who can flip a HomePass to RFS, which is commercially load-bearing)
- KE addressing convention: ODD/EVEN house numbering relative to road, building # captured in field, uniqueness blocked at system level, clone with apartment-change pattern
- Mixed-tech rule: GPON / HFC mix within a TechRegion; access_technology captured per HomePass
- Tech Region drives contractor assignment for WOs
- ~6 franchises in KE; Nairobi has ~8–9 contractors mapped via TechRegions
- Number entered = active termination points (signal-provisioned), basis for contractor payment

What's NEW in V3 / not in TZ Sophix and not clean in BH today:

1. **Whole module is new.** TZ Sophix doesn't have these entities at all — coverage is implicit in subscription captures and doesn't have a dedicated catalog. V3 creates a homepass module from scratch.
2. **Configurable Camunda workflow for status transitions.** BH KE relies on access control + manual discipline; TZ has no formal transition control. V3 routes every status change through a deployment-selected Camunda BPMN — strict maker-checker for deployments that want it (typically gating any flip to RFS, the commercially load-bearing status), passthrough for deployments not needing approvals, or any custom variant per deployment. The workflow IS the policy; there are no hardcoded transitions in DRL or in DB constraints. HOMEPASS-OP-01 (construction workflow, separate DD) calls into this same workflow as a child process.

3. **HomePassReachedSellable event for auto-notify queued leads.** Today, sales leads captured against an ineligible address are stored, but there's no automatic outbound when the HomePass becomes sellable. V3 emits an event (deployment-independent, workflow-path independent) that the lead-capture system consumes to drive the SMS/email.
4. **Bulk import as a clean API.** The current NS2 Schema dialog is broken; we rebuild as a versioned, validated CSV import endpoint with atomic batch semantics and a clear error report.
5. **Cross-module ref validation.** Today the Franchise field on the BH HomePass dialog is a free-text dropdown; nothing prevents an invalid franchise code. V3 enforces that Franchise and TechRegion exist and are ACTIVE at the time of HomePass save (independently — no parent-child constraint).
6. **Soft-delete with cross-module check.** Today a HomePass can in principle be edited / removed while customer services are bound to it. V3 prevents this by calling fulfillment REST.
7. **Standard event envelope.** TZ doesn't publish; KE BH doesn't publish either. V3 puts every catalog mutation on Kafka so the rest of the BSS (onboarding, fulfillment, field-ops, billing, cvm, search) stays in sync via cache invalidation.
8. **Structured network_path per HomePass — with ports.** BH captures Head End, Hub Number, Node, Drop Tag as a flat set of fields with no ordering and inconsistent port capture. V3 models the network nodes as an ordered chain with type / code / role / port, supporting both fiber-pre-deployed (KE: full path with ports known at HomePass time) and fiber-on-customer-install (TZ-style: port captured by field ops at install, appended to the path via a dedicated endpoint). The serviceManagementEndpoints derivation gives provisioning-gw and other gateways a single short blob {nodeCode, port} to address a specific subscriber line — without ports, fulfillment cannot operate.
9. **GeoJSON polygons on Franchise + TechRegion** (optional). BH has no concept of franchise / techregion boundaries beyond labels. V3 stores GeoJSON for the geographic footprint. v1 stores opaque JSONB; operations on the polygons (auto-infer TechRegion from coordinates, polygon-vs-point queries, validation) are deferred — but the data is there from day one for any module that wants to render a map.
10. **Many-to-many HomePass↔Franchise and HomePass↔TechRegion.** BH carries a single Franchise and a single Tech Region per HomePass. V3 supports a HomePass linked to multiple Franchises (commercial overlap, B2B vs B2C franchise coexistence) and multiple TechRegions. Migration imports BH single-values as 1-element lists; admins extend as needed.
11. **TechContractor as a first-class entity with skills.** BH has no explicit Contractor entity — install crews and support teams are implicit. V3 introduces TechContractor with a configurable skills[] array (drawn from the TechContractorSkill catalog: INSTALLATION, MAINTENANCE, SUPPORT, ...), and a tech_region_contractor many-to-many join saying which contractors serve which TechRegions. Skills belong to contractors (the people doing the work), not regions (the geography). RLM-CFG-01 exposes a GET /api/homepasses/{id}/eligible-contractors?skill=... lookup primitive; the actual WO routing logic — slot capacity, schedule, multi-hop role transfers (installer → design → installer back) — is owned by the future WO DD.
12. **Derived servicesSupported per HomePass.** Computed from network_path at save time. Answers "what can this HomePass physically carry?" — DATA, VOICE, IPTV_MULTICAST. A phone-only POTS customer is modeled as a HomePass with accessTechnology = COPPER and servicesSupported = ["VOICE"]. Used by onboarding for package-eligibility filtering and by cvm for service-mix segmentation. Cloud / OTT services (mobile data via partner SIM, streaming-app IPTV, partner-billed bundles) are NOT modeled here — they're handled by Service def serviceManagementKey = null on the PLM-CFG-01 side.
13. **HomePass status as a per-deployment configurable catalog.** BH and TZ both hardcode their status sets in the codebase. V3 makes homepass_status_code an admin-managed catalog with semantic flags (is_sellable, is_active, is_terminal, requires_approval_to_enter, triggers_lead_notification, blocks_soft_delete). Downstream behavior reads the flags, never literal code names: the eligible-sellable lookup, the soft-delete check, the lead-notification trigger, and the reference BPMN's approval gateway all consult the catalog. KE seeds with BH's full set (NPL/NSR/NSV/NSN/PH1/PH2/PH3/RFC/RFN/RFS/WAI/ACT); a new deployment seeds with the 6 default codes. Adding a status code is now an admin operation, not a code release.
14. **Hierarchical structured address.** BH carries a single flat Address string plus a handful of broken-out fields; address corrections happen via WO free-text notes. V3 stores 8 admin/geographic levels (country, region, region_l1, region_l2, city, area, sub_area_1, sub_area_2) plus 4 street levels (road_name, building_number, building_name, apartment_number). Uniqueness is the full tuple. The structure works across deployments (KE uses County / Sub-County; SN uses Région / Département; admins fill the levels their geography actually needs). Address is mutable post-create (R-RLM-CFG-01-H-14) with a dedicated HomePassAddressCorrected event — WO module (separate DD) can call structured corrections rather than free-text notes.
15. **NetworkNode catalog formalizes BH's dropdown convention.** BH's Head End / Hub Number / Node fields are dropdown-driven (codes like RNA for RUNDA OLT, TK7 for FRT 12-47) but the catalog is implicit, scattered across BH's generic Lists admin. V3 makes network_node a first-class entity with code, type, name, optional parent_node_code

(encodes the Wananchi convention of "from Node 104-25 you walk to its parent 104 and you know it's behind fiber node 104"), and JSONB metadata for future vendor/model/GPS data. HomePass network_path references catalog codes (R-RLM-CFG-01-H-15). Forward path: a future network-topology DD may auto-populate the catalog from OLT auto-discovery feeds (Wananchi runs `ma logs.wananchi.com/gpon_auto_find/` polling OLTs every 5 minutes).

16. **HouseType as a configurable catalog.** BH carries a controlled dropdown of MDU classes (M2M, S1H, M1H, etc.) used downstream for sales prioritization. V3 makes this a proper catalog with admin CRUD instead of relying on BH's generic Lists admin.
17. **Clone HomePass.** BH's "Clone From" lets you copy an address; V3 generalizes this to clone a full HomePass (including network_path, refs, address fields) with one field varying per clone (typically apartment_number). 48 apartments in one building become one create + one clone-call-with-count, not 48 individual creates.
18. **Building metadata on the HomePass row.** The design map's blue building box carries num_apts and num_floors (e.g., FINWOOD APARTMENTS 48/13). V3 captures these as building_num_floors and building_num_apartments on the HomePass row — same values across all apartments in the building (admin fills once, clone propagates). Used for capacity reporting and building-level dashboards. Per the architectural decision in this DD, building data is denormalized on each HomePass rather than split into a separate Building entity (a building without HomePasses is not meaningful in this system).
19. **RoE date and link captured as audit data.** BH does not carry the Right-of-Entry signing date or signed-document link as structured fields on HomePass — they live in adjacent spreadsheets, PDF archives, and the construction-tracking workbook. V3 adds roe_signed_date and roe_document_link as denormalized columns on the HomePass row (same values across all apartments in the building). The full RoE lifecycle — signing workflow, owner contact, renewal, expiry — stays in a future RoE / construction-survey DD; this DD just records the fact-of-having-RoE for audit and reporting (e.g., "show me all sellable HomePasses whose RoE was signed >24 months ago and may need renewal"). R-RLM-CFG-01-H-16 keeps the RoE state coherent with the HomePass status (an NSV HomePass can't have a signed date).
20. **Optional GIS integration via internal geo-client sub-module.** Strictly additive — every existing HomePass field still works without GIS. When GIS is configured (default adapter targets Google Maps Platform, Wananchi's current GIS), three optional features unlock: (a) **address autocomplete** on the create-HomePass form (admin types "Wood Ave", picks from a dropdown, structured fields and coordinates auto-fill), (b) **bidirectional enrichment** (POST /enrich-from-geo reverse-geocodes coordinates to admin fields OR forward-geocodes admin fields to coordinates; can be batched at bulk-import time), and (c) **map view** in the Backoffice (HomePasses as colored pins by status, TechRegion / Franchise polygons as overlays, clustered at low zoom). The adapter pattern abstracts the provider so swapping Google Maps for ArcGIS / GeoServer / PostGIS / a custom service is an adapter-class change behind geo-client, not module-wide change. The GIS capability lives inside the homepass module — not a separate foundation gateway — because RLM-CFG-01 is the only consumer in V3. If a second module (Equipment, WO, Construction) later needs the same capability, geo-client will be promoted to a geo-gw foundation at that point. A new optional google_place_id column on homepass captures the stable Place ID when sourced from autocomplete. R-RLM-CFG-01-H-17 (coherence warning, advisory) and R-RLM-CFG-01-H-18 (Place ID immutable once set) added. PDF design maps are handled via the existing map_link column (URL reference) plus incremental digitization into the network_node catalog as work happens — the two coexist, with PDFs as visual ground truth and the catalog as structured queryable data. Automated PDF-to-GeoJSON conversion is explicitly out of scope for V3.

C — Gap & bridge

Migration approach: greenfield with one-time BH import

The Wananchi data lives in Broadhub. There's no TZ Sophix data to migrate (TZ has no homepass entities). Approach:

1. **Build the homepass module greenfield** with the schema, rules, events, APIs from this DD. New module, new DB, new Kafka topics, new KJAR.
2. **One-time bulk import from BH:** extract the Homes Passed table from BH (CSV per Franchise), map to V3 fields, load via the new `/api/homepasses/bulk-import` endpoint. Done per Franchise; total ~hundreds of thousands of rows across KE.
3. **Field mapping for the import (BH → V3):**
 - BH Acct # → not imported (BH internal id)
 - BH Address (free-text composite, e.g., WOOD AVENUE 12 FINWOOD APARTMENTS # APT 18 KILIMANI NAIROBI) → **parsed into the address hierarchy**. The importer uses heuristics (regex + per-deployment patterns) plus the BH search-dialog field structure (Road Name, Building #, Building Name, Apt, Area, City) to populate road_name,

building_number, building_name, apartment_number, area. Where BH stored a single composite string, the importer best-effort splits it; rows where parsing is ambiguous are flagged in the validation report for admin review before commit.

- BH Apartment (from Misc section) → apartment_number (authoritative over any parsed value)
- BH Building (the standalone Building field on the dialog) → building_name (authoritative over the parsed value)
- **City** in BH (e.g., NAIROBI) → typically maps to region_l1 (Nairobi County) or to area depending on deployment convention — confirm with the data ops team per country at migration time
- **Country** is set per-import-batch (the batch metadata declares the deployment country, e.g., KE); not from a BH field
- region, region_l2, sub_area_1, sub_area_2 may not be populable from BH (BH doesn't carry these levels). Imported as NULL; admins can backfill if/when a regional reference exists
- BH Property Type (RES / COM) → property_type
- BH Owner occupied → owner_occupied
- BH Tech Region → resolved to a single TechRegion ref, inserted as a 1-element techRegionRefs[]. Admins can ADD additional TechRegion refs post-import as needed.
- BH Franchise → resolved to a single Franchise ref, inserted as a 1-element franchiseRefs[]. Admins can ADD additional Franchise refs post-import for overlap zones.
- BH Tech (technology) + technology flag → access_technology (GPON or HFC)
- BH provisioning fields Head End, Hub Number, Node, Drop Tag, and any Port/Slot fields → assembled into network_path with ports. **Each code must already exist in the pre-seeded network_node catalog** (R-RLM-CFG-01-H-15) — see seed order below. Mapping shape:
 - For GPON rows: nodes [{type:"OLT", code:<Head End>, role:"service_management", port:<port if BH captured it; else null>}, {type:"FAT", code:<Hub Number or Node>, role:"termination", port:<Drop Tag if it represents a port>}] (a 2-node simplified path; real KE deployments may have splitters in between but BH doesn't capture them as separate fields — those are recovered later from the design map if needed)
 - For HFC rows: nodes [{type:"HEADEND", code:<Head End>, role:"service_management", port:null}, {type:"DISTRIBUTION_NODE", code:<Hub Number>, role:"passive", port:null}, {type:"AMPLIFIER", code:<Node>, role:"termination", port:<Drop Tag>}]
- captureMode = "PRE_INSTALLATION" for all KE rows (KE deploys pre-customer). For deployments where field ops captures ports during install (e.g., TZ), import sets captureMode = "AT_INSTALLATION" and leaves ports null — they will be filled via the network-path append endpoint during install.
- **BH port capture completeness is unconfirmed at design time.** Where BH has no port detail, V3 imports with port : null. Whether this is a significant or marginal gap depends on the actual state of BH data — the technical team should sample-audit before committing to a remediation plan. The Risks section documents this open question.
- service_management_endpoints and services_supported → derived from network_path at insert time, not from BH (for KE: most rows derive services_supported = ["DATA"] from OLT/HEADEND presence; voice-enabled HomePasses derive ["DATA", "VOICE"] when a VOIPSWITCH node is present)
- BH Outlets → outlets
- BH Status (full set: NPL, NSR, NSV, NSN, PH1, PH2, PH3, RFC, RFN, RFS, WAI, ACT) → status_code referencing the deployment's homepass_status_code catalog. The KE catalog is **pre-seeded with the full BH set** before HomePass import (see seed order below). The BH code is preserved verbatim in legacy_status_code for audit. For BH NSV rows specifically, not_serviceable_reason = "NO_ROE" is set on import to make the RoE-pending reason explicit.
- BH Floor → floor (free-text)
- BH House Type (M2M, S1H, M1H, etc.) → house_type_code referencing the deployment's house_type catalog (pre-seeded from BH's House Type dropdown values — see seed order)
- BH Perm Date, Survey Date → corresponding columns
- **RoE fields** (roe_signed_date, roe_document_link): BH does not carry these as structured fields in its HomePass form. They may exist in adjacent systems (RoE management spreadsheets, scanned PDF archives, the construction-tracking workbook). Best-effort backfill at migration: where the technical team can match BH HomePasses to RoE records by address + building, populate; otherwise NULL. For BH rows with status NSV (NO_ROE), both fields are

explicitly NULL (the absence IS the data point — RoE not yet signed). Going forward in V3, these are filled at HomePass-create time when the RoE is signed; the future RoE DD will manage the signing workflow and POST updates here.

- BH coordinates (if present, e.g., GPS strings from WO notes like 1°12'46.9"S 36°52'35.3"E) → latitude, longitude (parsed to decimal degrees)
- BH Map Code → map_code (preserved as free-text)
- **GIS enrichment at migration (optional)**: when GIS is configured at migration time AND a BH row has latitude/longitude populated, the import path can queue a background reverse-geocode call to fill missing admin-hierarchy fields. Per-row cost is one Google Maps API call (~\$0.005 at current pricing); batches of 10k rows cost ~\$50. Similarly, rows with full admin hierarchy but null coordinates can be forward-geocoded. google_place_id is filled from the geocoder response if available. This enrichment is best-effort: rows where the geocoder fails or returns ambiguous results import with whatever fields the CSV provided, NULLs preserved. The migration team can decide per-Franchise whether to run enrichment at import or defer to the post-hoc /enrich-from-geo admin endpoint.
- BH wizard Directions / Comments / Notes → corresponding columns

4. Order of seeding:

- Seed the **TechContractorSkill catalog** first (admin creates INSTALLATION, MAINTENANCE, SUPPORT, etc.)
 - Seed **TechContractors** with their skills (the install/support companies operating in the deployment)
 - Seed **NetworkNodes**: the controlled list of OLTs, FATs, Splitters, Headends, etc. used in this deployment. Seed by exporting BH's Head End / Hub Number / Node dropdown values and mapping each to a NetworkNode entry with the right type. Establish parent_node_code chains where the convention is known (e.g., each FAT references its OLT, e.g., 104-25 → parent 104 per the design map). HomePass bulk-import will fail any row whose path references an unknown node code, so the catalog must be complete first.
 - Seed the **HomePassStatusCode catalog** with the deployment's status set. For Wananchi KE, seed all BH codes (NPL, NSR, NSV, NSN, PH1, PH2, PH3, RFC, RFN, RFS, WAI, ACT, RETIRED) with appropriate semantic flags: NPL = is_initial; RFS and RFN = is_sellable (RFN is sellable-but-capacity-exhausted, still sellable in principle for the queue); RFS = triggers_lead_notification; ACT = is_active + blocks_soft_delete; WAI = blocks_soft_delete; RETIRED = is_terminal; RFS, WAI, ACT = requires_approval_to_enter. For a fresh deployment, seed the 6-default-code set instead.
 - Seed the **HouseType catalog** with the BH House Type dropdown values: M2M (MDU C1 Medium), S1H (Single-family), M1H (MDU AB High), M0M (MDU Not Surveyed Medium), etc. — direct 1:1 copy of BH list values.
 - Seed Franchises (~6 ACTIVE Franchises for KE)
 - Seed TechRegions (~50+ across KE, each created as DRAFT)
 - Assign Contractors to TechRegions via the join endpoint
 - Activate TechRegions (R-RLM-CFG-01-T-7 fires — requires ≥1 active contractor assigned)
 - Deploy the Camunda BPMN for homepass-status-transition (strict reference BPMN for deployments wanting maker-checker, passthrough or custom for others). The BPMN gates on requires_approval_to_enter flag — no code names hardcoded.
 - Bulk-import HomePasses last, in batches per (legacy) Franchise. Each row's franchiseRefs[] and techRegionRefs[] come in as 1-element lists; address fields are parsed from BH's composite address string per the field mapping above; statusCode and houseTypeCode reference the pre-seeded catalogs.
5. **has_been_sellable initialization**: set to true for all imported HomePasses whose current status code is_sellable OR is_active (they've reached a sellable status at some point in their history); set to false for everything else. This prevents spurious HomePassReachedSellable events when later transitions happen for already-historically-sellable HomePasses. Status semantic flags are NOT denormalized onto the HomePass row — only the FK status_code is stored, with flags resolved via JOIN at query time.
6. **Status history**: no per-row history imported from BH (BH has Home Change History but mapping is complex and out of scope). Going forward, every transition in V3 is a Camunda process instance whose history (proposedBy, decidedBy, timestamps, variables) lives in Camunda's history tables. Querying /api/homepasses/{id}/transitions projects this on demand.
7. **Construction workflow stays in BH for now**: HOMEPASS-OP-01 (the construction-request → approval → contractor → MDU → design workflow) is deferred. Until then, the technical team continues in BH and ancillary tools, and only the resulting **approved** HomePass is created in V3 (via the new manual create endpoint, or — when HOMEPASS-OP-01 ships — via a Camunda workflow that calls into this DD's status-transition workflow).

Cross-DD coordination

This DD owns several primitives that other DDs depend on. The contracts below are the surface area; the consuming DDs own everything beyond.

Concern	What RLM-CFG-01 provides	What the consuming DD owns
WO routing (future WO DD)	GET /api/homepasses/{id}/eligible-contractors?skill=X lookup, returning contractors that (a) serve a TechRegion linked to this HomePass and (b) carry the requested skill	Mapping WO types to required skills; slot/period capacity scheduling (as observed in BH's role-based slots with n/8 capacity per period: DISPATCH-MORNING, DISPATCH-AFTERNOON, DISPATCH-EVENING); multi-hop WO reassignment patterns (e.g., when an installer finds signal too weak at site, the WO hops to a design team to update the design, then back to a different contractor to remediate signal, then back to the installer to finish — observed at Zuku); manual vs auto assignment; the WO entity itself with its lifecycle and notes (BH today captures install-time observations as WO free-text — for example, address corrections, FAT positions, technician identification — those notes are how field knowledge flows back). All of this is firmly in the WO DD's scope; RLM-CFG-01 just exposes the eligibility lookup.
Equipment / ONT binding (future Equipment DD)	HomePass.network_path terminal node (type = ONT) carries the ONT identifier as its code. The OLT-side service-management node carries the slot/PON/port assignment as its port. The service_management_endpoints derived map gives the gateway-ready {nodeCode, port} pair.	The Equipment entity itself (ONU/ONT records with serial, vendor, model, warranty, OLT chassis slot / PON / ONU number, etc.). The Equipment record's location IS the HomePass's terminal node — the Equipment DD will define the formal link (likely an equipmentRef field added later to network_path.nodes[i] for ONT nodes; out of scope here). The auto-discovery feed (Wananchi runs an OLT polling portal at malogs.wananchi.com/gpon_auto_find/ listing discovered ONT serials with F/S/P — Frame/Slot/Port) is the network's ground truth for what's physically connected; the Equipment DD or a future network-topology DD will reconcile discovered devices against HomePass terminal-node assignments.
Address corrections from the field (future WO DD)	PATCH /api/homepasses/{id} accepts address-field updates at any HomePass lifecycle stage (R-RLM-CFG-01-H-14), emits HomePassAddressCorrected	Today in BH, when a field crew arrives at an install and finds the address wrong, they document the correction in WO free-text notes (observed at Zuku — e.g., FAT 38-42 marked in note, technician contact recorded). In V3, the WO module will be able to call the structured HomePass address-correction PATCH on behalf of the field crew, replacing the free-text path. The trigger (which WO note types translate to an automatic correction vs which require admin review) is a WO-DD policy decision.
Subscription binding at ACT (existing fulfillment / subscription module)	GET /api/homepasses/eligible returns, for HomePasses already at ACT, the bound {subscriptionId, customerName} summary so sales-team UX shows "address already taken by X" rather than just hiding the row.	The subscription / fulfillment module owns the data; RLM-CFG-01 fetches via a cached cross-module read. The fetch path and TTL are infrastructure-level concerns.
Network-node auto-discovery (future network-topology DD)	network_node catalog admin-managed in v1	A future DD may consume the gpon_auto_find portal (or its equivalent) to auto-populate the catalog and flag drift (a HomePass whose terminal-node ONT serial no longer matches what the OLT reports). The network_node.metadata.JSONB field is the extension point for vendor / model / GPS / configuration parameters.
Sales Agent / Sales Rep (future CRM or onboarding DD)	RLM-CFG-01 carries Franchise refs (commercial geographic zones) on the HomePass; nothing else	Sales Agent / Sales Rep is a person/team within a Franchise, owned by CRM. BH today shows a Sales Agent dropdown on the HomePass form; in V3 the agent ↔ HomePass relationship is computed CRM-side at acquisition time (which agent's territory covers this address?) rather than denormalized onto the HomePass row. The future CRM/onboarding DD owns agent management, territory mapping, and commission attribution.

Concern	What RLM-CFG-01 provides	What the consuming DD owns
Right of Entry (RoE) (future RoE / construction DD)	RLM-CFG-01 carries <code>status_code = NSV</code> (or equivalent flagged code) with <code>not_serviceable_reason = "NO_ROE"</code> to indicate the RoE block. Once an RoE is signed, the HomePass row carries <code>roe_signed_date</code> (the signing date) and <code>roe_document_link</code> (URL to the signed document). These two fields are denormalized — same values across all HomePasses in the same building (one RoE covers all apartments). No RoE management or signing workflow lives here; we just record the fact-of-having-RoE for audit.	The RoE management system (initial phase of building survey — owner permissions, signed documents, expiry dates, renewal workflow, contact details for the owner, photos of the building, etc.) is a separate DD, likely part of HOMEPASS-OP-01 or its own building-survey DD. It produces transitions that move HomePasses out of NSV via the configurable status workflow, and POSTs <code>roe_signed_date / roe_document_link</code> updates to RLM-CFG-01 when the signing completes.
GIS / mapping (internal to this module via <code>geo-client</code> sub-area)	Storage of polygons (<code>franchise.boundary_geojson</code> , <code>tech_region.boundary_geojson</code>), per-HomePass coordinates and <code>google_place_id</code> , a <code>/api/homepasses/map</code> tile endpoint for map rendering, and the <code>geo-client</code> adapter for enrichment via POST <code>/api/homepasses/{id}/enrich-from-geo</code> . All integration points are opt-in — RLM-CFG-01 works without GIS configured.	Owned within homepass module. The <code>geo-client</code> sub-area holds the adapter contract (geocode, reverse-geocode, autocomplete, place-details, contains), the API-key handling, and the cache table. Default adapter targets Google Maps Platform (Wananchi's current GIS); other adapters (ArcGIS, GeoServer, PostGIS-local) can be plugged in by implementing the same interface. If a second consumer module later needs GIS, <code>geo-client</code> will be extracted into a <code>geo-gw</code> foundation DD at that point. UI rendering (Google Maps embed in the Backoffice for the map view, MapBox GL or Leaflet for richer interactive maps) is a front-end concern handled by the Backoffice DD.
PDF design map digitization (future GIS-MIGRATION-01 / NETWORK-TOPOLOGY-01 DD, if and when prioritized)	The <code>map_link</code> column on homepass stores PDF URLs. The <code>network_node</code> catalog stores structured topology data digitized incrementally from PDFs as work happens. No PDF ingestion logic here.	If automated PDF-to-GeoJSON ever becomes a priority, a separate DD will own the conversion pipeline (vector parsing, raster digitization workflow), feeding into <code>network_node.metadata</code> and <code>tech_region.boundary_geojson</code> . Not part of V3 scope.

Dependencies

Dependency	Owner	Status
homepass module skeleton (new) with messaging schema, outbox/inbox workers, Drools client	homepass team	created by this DD
homepass-rules KJAR Maven project (new)	homepass team	created by this DD
fulfillment module REST endpoint for active-services-by-homepass count (used by R-RLM-CFG-01-H-7)	fulfillment team	■ pending (FUL-03 DD) — until then, R-RLM-CFG-01-H-7 short-circuits to "allow delete" with a warning
Lead-capture sub-area in onboarding to consume <code>HomePassReachedSellable</code> and trigger SMS/email via <code>notification-gw</code>	onboarding team	■ pending (likely ILM-CFG-01 or SIP-03 DD)
<code>notification-gw</code> (existing foundation gateway) for the outbound SMS/email	gateways team	■ from foundations
BH data export script per Franchise for the one-time bulk import	data ops	external

What's deferred to other DDs

- **Construction request workflow** (Commercial submits → Technical approves → Contractor → MDU formalities → technical design → V3 HomePass creation) → **HOMEPASS-OP-01**
- **Coverage check at acquisition** (the SIP-03 call that hits `/api/homepasses/eligible`) → **SIP-03** (this DD only defines the endpoint; the calling workflow lives in SIP-03)
- **Lead capture and auto-notify-on-RFS** (the consumer side of `HomePassReachedSellable`) → **onboarding-side DD** (likely ILM-CFG-01 or a new LEAD-CFG DD)
- **WO routing by TechRegion** (which contractor for which WO based on TechRegion) → **field-ops / EM-02 Contractor onboarding**
- **Map view in the Backoffice UI** for HomePasses by status — deferred to a UX-focused DD
- **Cross-country HomePass governance** (KE team validates UG/TZ HomePass requests as flagged in workshop) — workflow concern, lives in HOMEPASS-OP-01

- **Build certification artefacts (photos, signoff documents) and design map attachments** — deferred; tracked as document references on the HomePass once a document management foundation exists (HLD §18 deferred items)

Risks

- **Address normalization at bulk import.** BH has free-text Address and Apartment fields with inconsistent formatting (case, abbreviations, trailing whitespace). Without normalization, R-RLM-CFG-01-H-1 uniqueness will under-detect duplicates. Mitigation: a normalization step in BulkImportService (uppercase, trim, collapse whitespace) applied before uniqueness check; the normalized form is stored alongside the original for display.
- **Workflow definition correctness.** Transition validity now lives entirely in the Camunda BPMN. A misconfigured BPMN — e.g., one that allows NPL → RETIRED, or one that forgets the maker-checker user-task — would silently allow incorrect behavior. Mitigation: ship the strict reference BPMN as the KE default with a CI test suite that exercises every transition path against expected outcomes; require BPMN reviews on PR for deployments that fork it. The DD does NOT enforce a specific transition graph because different deployments legitimately need different graphs (e.g., a fully decommissioned-only flow for a winding-down deployment).
- **Maker-checker workflow flexibility.** The strict BPMN's gating policy (which transitions require approval, who can approve) is encoded in BPMN, not in code. If a deployment finds the rule too strict — for instance, wanting to auto-apply transitions that carry verified upstream evidence — that becomes a BPMN edit on a per-deployment basis, not a code release. The Status Workflow section documents the workflow contract every BPMN must honor; the policy choices on top are deployment-owned.
- **Workflow process instance hygiene.** Long-running pending approvals could pile up in Camunda's runtime. Mitigation: set a deployment-configurable timeout (e.g., 30 days) via a timer boundary event in the BPMN that triggers transition-rejected with reason "timed out"; an admin can re-propose if needed. The reference BPMN includes this timer.
- **Port-data quality at migration — to confirm with the team.** It is unclear at design time how complete BH's port capture is. The technical team should sample-audit a representative slice of BH HomePasses (across GPON and HFC, across WAI/ACT and earlier statuses) to determine the actual gap before any sub-project is committed. *If* it turns out a significant fraction of WAI/ACT HomePasses lack accurate ports, a reconciliation pass (comparing the imported network_path against the OLT NMS / design map) becomes necessary because fulfillment cannot operate the service without correct port-on-OLT data. For HomePasses still in NPL / NSN at import time, ports can be filled lazily as construction completes. The DD does not commit to a specific mitigation until the audit results are known.
- **HomePassReachedSellable event semantics for re-sellable.** R-RLM-CFG-01-H-6 fires the event only on the FIRST transition into a sellable status (latches via has_been_sellable). If a HomePass goes sellable → non-sellable → sellable (e.g., RFS → NSN → RFS for a re-design), the second flip does NOT re-fire. Confirm this is the desired semantics for the lead-notify use case — if Wananchi wants to re-notify on every entry into a sellable status, the rule changes to fire every time (and the latch goes away).
- **Bulk import atomicity at scale.** Importing hundreds of thousands of rows in one transaction is impractical. The "all-or-nothing per call" semantic in the API should be per-batch (e.g., 1000 rows per call). For the initial Franchise-level migration, the user uploads multiple batches; each batch atomic; failures in batch N do not roll back batch N-1.